

Sistema de Leaderboard para Juego en Unity

Alumno: Juan Bernáldez Pereda -72231203W

Fecha: 28 de Septiembre del 2024

Proyecto de Desarrollo de Aplicaciones Multiplataforma

Curso académico 2024/2025

Sumario

Proyecto: Sistema de Leaderboard para Juego.....	3
1. Resumen.....	3
2. Introducción.....	4
3. Objetivos.....	5
Objetivo General.....	5
Objetivos Específicos.....	5
4. Palabras Clave.....	5
5. Análisis del Contexto.....	6
5.1. Innovación.....	8
6. Diseño.....	9
6.1. Arquitectura General.....	9
6.2. Backend (API).....	10
6.3. Dependencias Instaladas.....	22
6.4. Servicio de Mail para la Recuperación de Contraseñas.....	23
6.5. Explicación JWT para Autenticación.....	25
6.6. Configuración de CORS.....	27
6.7. Generacion Informes.....	29
6.8. Frontend (Cliente Unity).....	31
7. Casos de Uso de los Usuarios.....	50
7.1. Caso de Uso 1: Iniciar Sesión en la ventana de Login.....	50
7.2. Caso de Uso 2: Recuperar Contraseña en la ventana de Login.....	50
7.3. Caso de Uso 3: Cambiar a la ventana de Registro desde la ventana de Login.....	50
7.4. Caso de Uso 4: Registrar una Cuenta en la ventana de Registro.....	51
7.5. Caso de Uso 5: Cambiar a la ventana de Login desde la ventana de Registro.....	51
7.6. Caso de Uso 6: Registrar una nueva Puntuación al Terminar la Partida.....	52
7.7. Caso de Uso 7: Consultar el Leaderboard General desde la ventana Menu.....	52
7.8. Caso de Uso 8: Ver el Perfil del Jugador desde la ventana Leaderboard.....	53
7.9. Caso de Uso 9: Actualizar Perfil del Jugador desde la ventana Leaderboard.....	53
7.10. Caso de Uso 10: Añadir Amigos.....	54
7.11. Caso de Uso 11: Aceptar Solicitudes de Amistad.....	55
7.12. Caso de Uso 12: Rechazar Solicitudes de Amistad.....	56
7.13. Caso de Uso 13: Cerrar sesión desde pantalla Menu.....	56
7.14. Caso de Uso 14: Eliminar una Puntuación (Administración).....	57
7.15. Caso de Uso 15: Reinicio del Sistema de Puntuaciones al Final de una Temporada (Administración).....	57
7.16. Caso de Uso 16: Reinicio del Sistema de Jugadores (Administración) (Para pruebas).....	58
8. BD game.....	59
8.1. Base de Datos game.....	59
8.2. Tablas y Atributos.....	59
8.3. Relaciones y Dependencias.....	61
9. Diseño de la interfaz.....	62
9.1 Implementación de las pantallas con funcionalidad parcial.....	68
9.2 Diagramas.....	68
Diagrama Entidad-Relación.....	70
Modelo Relacional.....	72
Modelo normalizado.....	74
Modelo Normalizado Final.....	75

9.3 Planificación.....	77
10. Ejecución del Proyecto (Puesta en marcha).....	77
11. Explotación del Proyecto.....	85
1. Despliegue del Backend:.....	85
2. Explotación del Cliente Unity:.....	87
12. Gestión económica o plan de empresa.....	91
12.1. Presupuesto estimado:.....	92
12.2. Fuentes de financiación:.....	92
12.3. Proyección de ingresos:.....	92
12.4. Plan de sostenibilidad:.....	92
12.5. Riesgos financieros:.....	93
12.6. Análisis de costes-beneficios:.....	93
13. Conclusiones y valoración personal.....	93
13.1. Conclusiones:.....	93
13.2. Valoración personal:.....	94
14. Bibliografía.....	95
15. Anexos.....	95
Anexo A: Diagramas del Sistema.....	95
Anexo B: Especificaciones Técnicas.....	95
Anexo C: Código Fuente Relevante.....	96
Anexo D: Manual de Usuario.....	96
Anexo E: Documentación Adicional.....	96

Proyecto: Sistema de Leaderboard para Juego

1. Resumen

El presente proyecto tiene como objetivo desarrollar un sistema de **Leaderboard** para un juego, el cual permite gestionar las puntuaciones de los jugadores de forma eficiente y segura. El sistema incluye un servidor desarrollado en **Flask** que expone una **API RESTful**, encargada de conectarse con una base de datos **PostgreSQL** para almacenar y gestionar las puntuaciones. La **API** implementa medidas de seguridad mediante el uso de **JWT** (JSON Web Tokens) para garantizar un acceso controlado. Además, permite realizar diversas operaciones, como ordenar las puntuaciones de forma ascendente o descendente, buscar la puntuación de un jugador por su nombre, ver y actualizar información de tu perfil de usuario así como la gestión de privacidad y amigos permitiendo decidir si queremos que nuestra puntuación sea privada, publica o solo visible para amigos, también incluye un mecanismo de seguridad **CORS** por si en un futuro se necesitase obtener recursos de un dominio diferente al nuestro, ademas, implementa la generación de **informes** mediante un programa en **java** que usa la librería **jasperreport** y un servicio **mail** en la **API** para recuperar la contraseña en caso de olvido.

El cliente, implementado en **Unity**, interactúa con la **API** para enviar y recuperar puntuaciones, asegurando la correcta comunicación entre el juego y el servidor. En resumidas cuentas lo que el proyecto busca es ofrecer una interfaz sencilla y funcional para consultar las puntuaciones de

manera ordenada desde cualquier cliente. Esta interfaz podrá ser integrada junto con su funcionalidad en juegos desarrollados con **Unity**.

2. Introducción

El uso de sistemas de Leaderboard en los videojuegos ha cobrado una gran relevancia en los últimos años, dado que permiten a los jugadores comparar su rendimiento con otros usuarios, fomentando la competitividad y mejorando la experiencia de juego. Sin embargo, implementar sistemas de Leaderboard eficientes, seguros y escalables sigue siendo un desafío para los desarrolladores, especialmente cuando se manejan grandes volúmenes de datos y se requiere asegurar la integridad y privacidad de la información de los usuarios.

Este proyecto tiene como objetivo diseñar e implementar una solución completa de backend y frontend que permita almacenar, recuperar y gestionar las puntuaciones de los jugadores de manera eficiente y segura. Para ello, el servidor se ha desarrollado usando **Flask**, un framework ligero y flexible en Python. La elección de Flask se debe a su facilidad de uso, su capacidad para crear APIs RESTful de forma rápida y su amplia comunidad de soporte, lo que asegura una curva de aprendizaje manejable y herramientas suficientes para manejar los requisitos del sistema. Además, Python es ampliamente utilizado en entornos académicos y profesionales por su versatilidad y simplicidad, lo que lo convierte en una elección adecuada para el desarrollo de este tipo de proyectos.

Por otro lado, los datos se persisten en una base de datos **PostgreSQL**, una solución robusta y de código abierto reconocida por su rendimiento y capacidad para manejar datos estructurados con integridad referencial. **PostgreSQL** ha sido seleccionado debido a su capacidad para gestionar grandes volúmenes de datos de manera eficiente y sus características avanzadas, como soporte para transacciones complejas, concurrencia, y extensiones, que la convierten en una base de datos ideal para este tipo de aplicaciones que requieren alta fiabilidad y consistencia en los datos.

En cuanto al cliente, este se ha desarrollado utilizando **Unity**, uno de los motores de videojuegos más populares en la industria. Unity ha sido elegido por su versatilidad, su amplia adopción en el desarrollo de videojuegos y su soporte multiplataforma, lo que facilita la integración del sistema de **Leaderboard** en cualquier tipo de juego. Además, su compatibilidad con C# y la disponibilidad de bibliotecas para interactuar con **APIs RESTful** lo hacen especialmente adecuado para la comunicación eficiente con el backend.

El cliente en Unity interactúa con la API para enviar las puntuaciones, mostrarlas a los usuarios y ofrecer funcionalidades adicionales, como la actualización del perfil del jugador la gestión de privacidad y amigos

Este proyecto se desarrolla en un contexto académico, como un Trabajo de Fin de Grado (TFG), y busca resolver una problemática común en la industria de los videojuegos: la necesidad de contar con un sistema de **Leaderboard** flexible, seguro y adaptable que pueda ser integrado fácilmente en cualquier juego. La elección de estas tecnologías se basa en su popularidad, su capacidad para manejar los requisitos específicos del proyecto y su facilidad de integración en entornos académicos y profesionales.

En esta primera fase, se describirán los objetivos generales y específicos del proyecto, el análisis del problema a resolver, y la justificación de las decisiones tomadas, tanto a nivel técnico como funcional. De esta forma, se busca demostrar cómo el proyecto puede aportar valor a la comunidad de desarrolladores de videojuegos y ofrecer una solución robusta para gestionar los rankings de jugadores, el proyecto incluye servidor mail para restablecer la contraseña, generación de informes

3. Objetivos

Objetivo General

Desarrollar un sistema completo de Leaderboard que permita la gestión de puntuaciones para un videojuego, utilizando tecnologías modernas de backend (Flask, PostgreSQL) y frontend (Unity).

Objetivos Específicos

- Diseñar una **API RESTful** en **Flask** que permita agregar, actualizar consultar y eliminar puntuaciones de los jugadores en tiempo real.
- Implementar una base de datos en **PostgreSQL** que almacene de manera persistente las puntuaciones y los nombres de los jugadores.
- Desarrollar un cliente en **Unity** que interactúe con la **API** para enviar, actualizar y consultar puntuaciones en tiempo real.
- Garantizar la seguridad y la integridad de los datos en las operaciones realizadas entre el cliente y el servidor.
- Asegurar que la solución sea escalable y fácil de mantener.

4. Palabras Clave

Palabras clave proyecto sistema de Leaderboard:

1. **Leaderboard**
2. **Juego**
3. **API RESTful**
4. **Flask**
5. **PostgreSQL**
6. **JWT (JSON Web Tokens)**
7. **Seguridad**
8. **Privacidad**
9. **Amigos**
10. **Generación de informes**
11. **Java**
12. **JasperReports**
13. **CORS**
14. **Unity**
15. **Autenticación**
16. **Puntuaciones**
17. **Desarrollo en Python**

5. Análisis del Contexto

El sistema de **Leaderboard** se ha convertido en una característica esencial en muchos videojuegos modernos, ya que fomenta la competitividad entre los jugadores, aumentando la retención y el compromiso con el juego. Los sistemas de rankings permiten a los jugadores comparar su rendimiento con otros, lo cual mejora la experiencia y la motivación para seguir jugando. Este tipo de sistemas también juega un papel fundamental en la creación de comunidades en línea, donde los jugadores buscan destacarse entre sus pares. Actualmente, existen diversas soluciones comerciales para la gestión de puntuaciones, como PlayFab o GameSparks, que ofrecen plataformas listas para usar. Sin embargo, desarrollar un sistema propio tiene varias ventajas, entre las que destacan el control total sobre la infraestructura, la capacidad de personalizar la solución según las necesidades del juego y la posibilidad de integrarlo de manera más eficiente con otros sistemas internos.

Este proyecto se enmarca en la creación de un videojuego de pequeña escala, dirigido a una comunidad de jugadores que buscan una experiencia personalizada y directa. Para ello, se ha diseñado un sistema de puntuaciones que permitirá a los jugadores ver sus logros y compararlos con los de otros. Además, la capacidad de expandir este sistema a medida que el juego crezca y se añadan nuevas características, como nuevos modos de juego o contenido adicional, hace que la elección de una arquitectura flexible sea crucial.

El contexto tecnológico incluye el uso de **Flask** para desarrollar una **API RESTful** que se conecta a una base de datos **PostgreSQL**. Flask se elige por su ligereza, facilidad de uso y capacidad para crear APIs rápidamente, lo que permite desarrollar una solución eficaz sin añadir complejidad innecesaria. PostgreSQL se ha seleccionado por su robustez, fiabilidad y facilidad para manejar grandes volúmenes de datos. Estas características son esenciales para la escalabilidad del sistema, especialmente si se integran nuevas funcionalidades o si se aumenta el número de jugadores.

La elección de **Unity** como motor para el desarrollo del juego responde a su amplia adopción en la industria, su capacidad para crear experiencias interactivas multiplataforma y su facilidad de integración con sistemas backend a través de **APIs RESTful**. Unity permite una implementación eficiente de la interfaz gráfica del cliente y asegura la compatibilidad con diversas plataformas, lo que hace que el sistema de **Leaderboard** pueda ser utilizado en una amplia gama de dispositivos y sistemas operativos.

Análisis de la competencia

En el ámbito de los sistemas de Leaderboard, existen varias soluciones comerciales que se ofrecen como servicios listos para usar. PlayFab y GameSparks son dos de las principales plataformas que proporcionan servicios de backend como puntuaciones de jugadores, tablas de clasificación y gestión de datos en tiempo real. Estas plataformas están bien establecidas y cuentan con una amplia base de usuarios. Sin embargo, su principal desventaja radica en la falta de personalización para

necesidades específicas de cada juego, además de que el coste de estas plataformas puede resultar elevado en proyectos pequeños o medianos. Además, en muchos casos, se pierde el control total sobre los datos y la infraestructura.

Por otro lado, un sistema propio como el que se propone en este proyecto ofrece la ventaja de una personalización total, sin depender de plataformas de terceros. Al tener control sobre la infraestructura y los datos, se pueden implementar medidas de seguridad más ajustadas a las necesidades del juego y gestionar mejor la escalabilidad, lo que permite una evolución del sistema conforme crezca la base de usuarios.

Análisis DAFO

- **Fortalezas:**

- Control total sobre la infraestructura y la personalización.
- Escalabilidad para futuros crecimientos del sistema.
- Flexibilidad para adaptarse a las necesidades específicas del juego.

- **Oportunidades:**

- El sistema puede integrarse fácilmente con nuevos juegos desarrollados en Unity.
- La creciente popularidad de los juegos en línea y la gamificación crea una demanda constante de sistemas de Leaderboard efectivos.

- **Debilidades:**

- Requiere inversión de tiempo en desarrollo y mantenimiento.
- Puede ser necesario un equipo técnico para mantener la infraestructura y garantizar la seguridad.

- **Amenazas:**

- La competencia de plataformas comerciales establecidas como PlayFab y GameSparks, que ofrecen soluciones más rápidas pero menos personalizables.
- Riesgo de escalabilidad a medida que crezca la base de usuarios si no se gestionan adecuadamente los recursos.

5.1. Innovación

El proyecto del **Sistema de Leaderboard para Juego en Unity** incorpora varias innovaciones que lo distinguen de las soluciones existentes en el mercado:

1. Funcionalidades:

- **Filtros** : Los usuarios pueden filtrar el Leaderboard por usuarios, y también pueden ordenar las puntuaciones de mayor a menor o de menor a mayor.
- **Privacidad** : Los usuarios pueden gestionar su privacidad
- **Amigos** : Los usuarios pueden gestionar amigos
- **Funcionalidades de Admin:**

Generación de informes (de manera externa a la api directamente sobre la BD usando jasperreports y java), para ello, el **script** en **java JasperReportExecutor** conecta con la **BD game** mediante el **usuario admin** que se genera en **postgre** al cargar nuestro **script postgresql**, este **script postgresql** también genera el usuario **client** en **postgre** el cual es usado para la conexión con la **BD game** desde el backend **API Flask**

Uso de los endpoints de borrado (los que requieren rol de admin) para esto usamos un usuario de la **api (tabla jugadores)** que tenga el **rol** de **admin** y además debemos tomar el **token** de **autenticación** del usuario (**access_token**) y pasarlo cuando llamemos al endpoint mediante un curl o postman.

Debido a que desde el lado del cliente no hay forma de llamar a estos endpoints, el sistema ofrece mayor seguridad, ya que la única forma de llamarlos es mediante curl o postman.

2. Escalabilidad y Mantenibilidad:

- **Arquitectura Modular:** Diseño del sistema con una arquitectura modular que facilita la incorporación de nuevas funcionalidades y la adaptación a diferentes tipos de juegos sin necesidad de reestructuraciones significativas. Se ha mantenido una estructura modular en el lado del cliente, funciones separadas por scripts.
- **Microservicios:** En el lado del servidor(API) se realiza una implementación de microservicios en el backend para manejar diferentes aspectos del Leaderboard de manera independiente, mejorando la escalabilidad. Se han mantenido los servicios separados por endpoints pero se han centralizado en un mismo archivo app.py

3. Seguridad Mejorada:

- **Autenticación y Autorización Robustas:** Uso de protocolos avanzados de seguridad para garantizar que solo usuarios autenticados puedan acceder y modificar las puntuaciones. *Falta implementar token de refresco*

6. Diseño

El diseño del **Sistema de Leaderboard para Juego en Unity** se divide en dos principales componentes: el **Backend** y el **Frontend**. A continuación, se detalla la arquitectura y los componentes clave de cada uno.

Cliente Unity (Frontend)

Motor de juego: Unity
Lenguaje: C#
Manejo de JSON: SimpleJSON
Interfaz gráfica:
- Pantalla de Leaderboard con opciones de filtrado
- Perfil del Jugador mostrando estadísticas y logros

Backend API (Flask)

Framework: Flask
Base de datos: PostgreSQL
Autenticación: JWT (JSON Web Tokens)
Seguridad: CORS

API RESTful:

- CRUD para puntuaciones: Crear, Leer, Actualizar, Eliminar
- Endpoints principales:
/puntuaciones POST para agregar puntuaciones en la base de datos
/puntuaciones GET para obtener puntuaciones con paginación y filtros
/puntuaciones/{nombre} GET para buscar por nombre de jugador
/puntuaciones/{nombre} PUT para actualizar por nombre de jugador
- Endpoints que no se usan desde la interfaz de unity, solo Admin:
/puntuaciones/{nombre} DELETE para eliminar puntuaciones de la base de datos
/puntuaciones DELETE para eliminar todas las puntuaciones de la base de datos

Seguridad

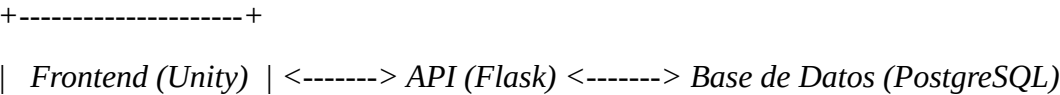
Autenticación: JWT
CORS configurado para orígenes autorizados

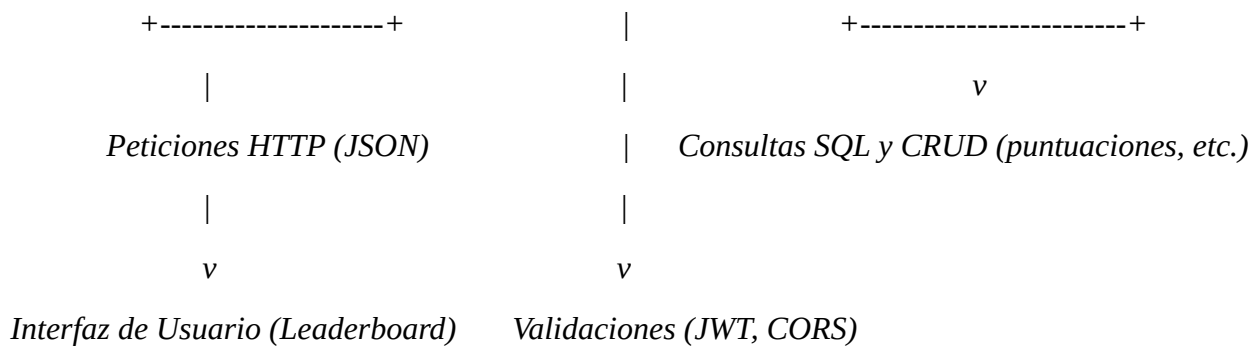
Base de Datos (PostgreSQL)

Almacena los datos de las puntuaciones de los jugadores
Esquema de puntuaciones:
nombre: string
valor: int

B.1. Configuración del Servidor Flask

6.1. Arquitectura General





A.1. Diagrama de Arquitectura General

6.2. Backend (API)

- **Framework:** Flask
- **Base de Datos:** PostgreSQL
 - **ORM:** SQLAlchemy
- **API RESTful:** Diseñada para manejar las operaciones CRUD (Crear, Leer, Actualizar, Eliminar) sobre las puntuaciones.
- **Seguridad:**
 - **JWT (JSON Web Tokens)** para autenticación segura.
 - **CORS (Cross-Origin Resource Sharing)** configurado para permitir solo orígenes autorizados.

Componentes Principales:

1. API Endpoints(app.py):

Autenticación y gestión de jugadores :

POST /jugadores/register

- **Descripción:** Permite registrar un nuevo jugador.
- **Parámetros:**
 - **nombre (string):** Nombre del jugador (máximo 12 caracteres).
 - **password (string):** Contraseña del jugador.
 - **email (string):** Correo electrónico único.
 - **numtelefono (string):** Número de teléfono (9 dígitos).
 - **rol (string, opcional):** Rol del jugador, por defecto "jugador" (No configurable desde el cliente, para crear un usuario administrador se debe introducir "admin" en el campo rol mediante un INSERT por el Administrador).
- **Respuesta:**

- 201: Registro exitoso. Retorna JSON con el ID y nombre del jugador.
- 400: Faltan parámetros o validación fallida.
- 500: Error interno del servidor.

Servicio de Correo Electrónico para la Recuperación de Contraseñas

POST /jugadores/send-reset-email

- **Descripción:** Este endpoint envía un correo electrónico con un enlace para restablecer la contraseña del jugador. El enlace contiene un token JWT que será utilizado en el siguiente paso del proceso.
- **Parámetros:**
 - email (string): Dirección de correo del jugador.
- **Respuesta:**
 - Código **200**: Si el correo se envió exitosamente con el enlace de restablecimiento.
 - Código **400**: Si no se proporcionó el correo o el correo no está registrado en la base de datos.
 - Código **500**: Si ocurrió un error al enviar el correo.
 - **Respuesta en caso de éxito:**
 json con los datos:

```
"message": "Correo enviado exitosamente"
```
 - **Descripción adicional:** Al enviar este correo, el jugador recibirá un enlace con un token JWT válido durante un tiempo limitado, que le permitirá acceder a la página de restablecimiento de contraseña.

GET /jugadores/reset-password/<reset_token>

- **Descripción:** Permite al jugador acceder a un formulario donde podrá ingresar su nueva contraseña utilizando el token JWT válido recibido en el correo enviado por **POST /jugadores/send-reset-email**.
- **Parámetros:**
 - reset_token (string): Token JWT generado para el restablecimiento de la contraseña.
- **Respuesta:**
 - Código **200**: Si se presenta correctamente el formulario de restablecimiento de contraseña.
 - Código **400_3**: Si el usuario asociado al correo del token no existe en la base de datos.
 - Código **511**: Si el token ha expirado y no es válido.
 - Código **400_8**: Si el token es inválido.

POST /jugadores/reset-password/<reset_token>

- **Descripción:** Permite al jugador restablecer su contraseña utilizando un token JWT válido obtenido a través del enlace enviado en **POST /jugadores/send-reset-email**.
- **Parámetros:**
 - password (string): Nueva contraseña que el jugador desea establecer.
 - repeatpassword (string): Confirmación de la nueva contraseña.
- **Respuesta:**
 - Código **200**: Si la contraseña se actualiza correctamente en la base de datos.
 - Código **400_5**: Si alguno de los campos de contraseña (password o repeatpassword) está vacío.
 - Código **400_6**: Si las contraseñas proporcionadas no coinciden.
 - Código **400_3**: Si el usuario no existe (esto también puede ocurrir si el correo del token no coincide con un usuario registrado).
 - **Respuesta en caso de éxito:**
 - Página HTML con el mensaje:

```
html

<!DOCTYPE html>
<html>
<head>
  <title>Contraseña Restablecida</title>
</head>
<body>
  <h1>Contraseña restablecida exitosamente</h1>
  <p>Ahora puedes iniciar sesión con tu nueva contraseña.</p>
</body>
</html>
```

Este flujo permite al jugador restablecer su contraseña en dos pasos: primero, enviando un correo con el enlace que contiene el token, y luego, usando ese token en el endpoint de restablecimiento de contraseña.

POST /jugadores/login

- **Descripción:** Permite a un jugador iniciar sesión.
- **Parámetros:**
 - nombre (string): Nombre del jugador.
 - password (string): Contraseña del jugador.
- **Respuesta:**

- 200: Inicio de sesión exitoso. Retorna un token de acceso (JWT).
- 400: Faltan parámetros.
- 401: Credenciales inválidas.
- 500: Error interno del servidor.

Gestión de perfil:

GET /jugadores/perfil

- Descripción: Obtiene el perfil del jugador autenticado.
- Requiere: Autenticación con token (JWT).
- **Respuesta:**
 - 200: Perfil del jugador (JSON con ID, nombre, email y teléfono).
 - 404: Jugador no encontrado.

PUT /jugadores/perfil

- Descripción: Actualiza el perfil del jugador autenticado.
- Requiere: Autenticación con token (JWT).
- **Parámetros:**
 - nombre (opcional): Nuevo nombre del jugador.
 - email (opcional): Nuevo email del jugador.
 - numtelefono (opcional): Nuevo número de teléfono.
- **Respuesta:**
 - 200: Perfil actualizado exitosamente.
 - 400: Faltan parámetros.
 - 404: Jugador no encontrado.
 - 500: Error interno del servidor.

Gestión de privacidad:

GET /privacidad/int:jugador_id

- **Descripción:** Permite obtener el nivel de privacidad de un jugador específico.
- **Parámetros:**
 - jugador_id (int): ID del jugador cuya privacidad se desea consultar.
- **Respuesta:**
 - **Código 200:** En caso de éxito, devuelve el nivel de privacidad del jugador.
 - **Código 404:** Si no se encuentra el jugador o el registro de privacidad para ese jugador.
 - **Código 500:** Si ocurre un error en el servidor.

Respuesta en caso de éxito:

json con los datos:

```
"jugador_id": 1,
"nivel_de_privacidad": "publico"
```

POST /privacidad

- **Descripción:** Permite crear un nuevo registro de privacidad para un jugador.
- **Parámetros:**
 - jugador_id (int): ID del jugador.
 - nivel_de_privacidad (string): Nivel de privacidad del jugador, que debe ser uno de los valores definidos en el enum NivelPrivacidad ("publico", "privado", "amigos").
- **Respuesta:**
 - **Código 201:** En caso de éxito, se crea el registro de privacidad.
 - **Código 400:** Si faltan parámetros o si el jugador ya tiene un registro de privacidad.
 - **Código 404:** Si no se encuentra al jugador.
 - **Código 500:** Si ocurre un error en el servidor.

Respuesta en caso de éxito:

json con los datos:

```
"message": "Privacidad creada correctamente",
"jugador_id": 1,
"nivel_de_privacidad": "publico"
```

PUT /privacidad/int:jugador_id

- **Descripción:** Permite actualizar el nivel de privacidad de un jugador específico.
- **Parámetros:**
 - jugador_id (int): ID del jugador cuya privacidad se desea actualizar.
 - nivel_de_privacidad (string): Nuevo nivel de privacidad del jugador.
- **Respuesta:**

- **Código 200:** En caso de éxito, se actualiza el nivel de privacidad.
- **Código 400:** Si falta el parámetro nivel_de_privacidad.
- **Código 404:** Si no se encuentra el jugador o el registro de privacidad para ese jugador.
- **Código 500:** Si ocurre un error en el servidor.

Respuesta en caso de éxito:

json con los datos

```
"message": "Privacidad actualizada correctamente",
"jugador_id": 1,
"nivel_de_privacidad": "privado"
```

Gestión de amigos:

POST /amistades/enviar_solicitud

- **Descripción:** Permite enviar una solicitud de amistad entre dos jugadores.
- **Parámetros:**
 - nombre_receptor (string): Nombre del jugador al que se le envía la solicitud.
- **Respuesta:**
 - **Código 201** en caso de éxito (solicitud enviada).
 - **Código 404** si alguno de los jugadores no existe.
 - **Código 400** si ya existe una solicitud pendiente o ya son amigos.

Respuesta en caso de éxito:

json con los datos:
"mensaje": "Solicitud enviada"

POST /amistades/aceptar_solicitud

- **Descripción:** Permite aceptar una solicitud de amistad pendiente entre dos jugadores.
- **Parámetros:**
 - receptor_id (int): ID del jugador que recibe la solicitud.
 - solicitante_id (int): ID del jugador que envió la solicitud.
- **Respuesta:**
 - **Código 200** en caso de éxito (solicitud aceptada).
 - **Código 404** si no se encuentra una solicitud pendiente.
 - **Código 404** si alguno de los jugadores no existe.

Respuesta en caso de éxito:

json con los datos:
"mensaje": "Solicitud aceptada"

POST /amistades/rechazar_solicitud

- **Descripción:** Permite rechazar una solicitud de amistad pendiente entre dos jugadores.
- **Parámetros:**
 - receptor_id (int): ID del jugador que recibe la solicitud.
 - solicitante_id (int): ID del jugador que envió la solicitud.
- **Respuesta:**
 - **Código 200** en caso de éxito (solicitud rechazada).
 - **Código 404** si no se encuentra una solicitud pendiente.
 - **Código 404** si alguno de los jugadores no existe.

Respuesta en caso de éxito:

json con los datos:

```
"mensaje": "Solicitud rechazada"
```

GET /amistades/obtener_pendientes

- **Descripción:** Permite obtener las solicitudes de amistad pendientes para el jugador autenticado.
- **Parámetros:** Ninguno.
- **Respuesta:**
 - **Código 200** con la lista de solicitudes pendientes.
 - **Código 404** si no se encuentra el jugador autenticado.

Respuesta en caso de éxito:

json con el texto:

```
"pendientes": [  
  
  "nombre": "Nombre del solicitante",  
  "solicitante_id": 1,  
  "receptor_id": 2  
  
  ...  
]  
}
```

GET /amistades/obtener_amigos

- **Descripción:** Permite obtener los amigos aceptados del jugador autenticado.
- **Parámetros:** Ninguno.
- **Respuesta:**
 - **Código 200** con la lista de amigos aceptados.
 - **Código 404** si no se encuentra el jugador autenticado.

Respuesta en caso de éxito:

json con una lista de amigos incluidos sus datos

faltaría:

añadir la posibilidad de dejar de ser amigos de otro usuario mediante un botón

mostrar también las solicitudes enviadas y no solo las recibidas

las solicitudes enviadas se mostraran con el estado pendiente aceptado o incluso rechazado

Gestión de puntuaciones:

antes manteníamos dos endpoints para el post y el put de puntuaciones:

POST /puntuaciones

- Descripción: Permite agregar una nueva puntuación.
- Parámetros:
 - nombre (string): Nombre del jugador.
 - valor (int): Puntuación del jugador.
- Respuesta:
 - Código 201 en caso de éxito.
 - Código 400 si faltan parámetros.
 - Código 409 si el jugador ya existe.
- Respuesta en caso de éxito:
JSON con la nueva puntuación agregada.

PUT /puntuaciones/{nombre}:

- **Descripción:** Actualiza la puntuación de un jugador específico.
- **Parámetros:**
 - nombre (obligatorio): nombre del jugador cuya puntuación se actualizará.
 - Cuerpo de la solicitud (JSON):
 - valor (obligatorio): el nuevo valor de la puntuación.
- **Respuesta:**

- Código 200 en caso de éxito:
- Código 400 si faltan parámetros
- Código 404 si no se encuentra la puntuación para el jugador
- Código 500 si ocurre un error durante la actualización
- **Respuesta en caso de éxito:**

JSON con la puntuación del jugador especificado actualizada.

Ahora tenemos uno post que comprueba si la puntuación es mayor y si es así la actualiza :

POST /puntuaciones

- **Descripción:** Permite agregar o actualizar la puntuación de un jugador en el sistema.
- **Parámetros:**
 - **nombre** (string): Nombre del jugador (obligatorio).
 - **valor** (int): Puntuación del jugador (obligatorio).
- **Respuesta:**
 - **Código 201** en caso de éxito al agregar una nueva puntuación.
 - **Código 200** en caso de éxito al actualizar una puntuación existente (si la nueva puntuación es mayor que la anterior).
 - **Código 400** si faltan parámetros obligatorios (nombre o valor).
 - **Código 422** si la nueva puntuación no es mayor que la existente.
 - **Código 404** si no se encuentra un jugador con el nombre proporcionado.
- **Respuesta en caso de éxito:**
json con los datos de la puntuación unido al jugador correspondiente
- **Respuesta en caso de error (falta de parámetros):**
json con los datos:
"error": "El nombre del jugador y el valor de la puntuación son obligatorios"
- **Respuesta en caso de error (puntuación no mayor):**
json con los datos:
"error": "La nueva puntuación debe ser mayor que la actual"
- **Respuesta en caso de error (jugador no encontrado):**
json con los datos:
"error": "No se encontró un jugador con el nombre 'nombre_del_jugador'"

GET /puntuaciones

Descripción:

Este endpoint permite obtener una lista de las puntuaciones ordenadas por un campo específico con soporte de paginación y filtrado de privacidad. Se incluye una verificación de amistad entre el jugador autenticado y otros jugadores según la configuración de privacidad, Si el nivel de privacidad del jugador con la puntuación a mostrar es "amigos", solo se mostrarán las puntuaciones si existe una relación de amistad aceptada entre el jugador autenticado y el jugador consultado.

Parámetros opcionales:

- **order_by**: Campo por el cual ordenar las puntuaciones. Por defecto, se ordena por valor. Este parámetro no es modificable desde la interfaz de usuario.
- **order**: Orden en el que se deben presentar los resultados. Puede ser asc (ascendente) o desc (descendente). El valor por defecto es desc.
- **page**: Número de página a recuperar. Por defecto es 1, no modificable desde la interfaz de usuario.
- **per_page**: Número de puntuaciones por página. Por defecto es 15, no modificable desde la interfaz de usuario.

Respuestas posibles:

- **Código 200**: Éxito. Se devuelve la lista de puntuaciones ordenadas y paginadas.
- **Código 400**: Error en los parámetros de entrada, como valores no válidos en page o per_page.
- **Código 404**: No se encontraron puntuaciones que cumplan los criterios de privacidad.
- **Código 422**: Parámetro de ordenación no válido o el valor de order no es asc ni desc.
- **Código 500**: Error en el servidor al procesar la solicitud.

Respuesta de éxito:

json con las puntuaciones de los jugadores filtrando por los que tengan privacidad publica y los que tengan privacidad amigos y sean amigos de usuario solicitante

GET /puntuaciones/{nombre}**Descripción:**

Este endpoint permite obtener las puntuaciones de un jugador específico, filtradas por su nombre.

Los resultados también están sujetos a la configuración de privacidad del jugador. Si el nivel de privacidad es "amigos", solo se mostrarán las puntuaciones si existe una relación de amistad aceptada entre el jugador autenticado y el jugador consultado.

Parámetros:

- **nombre** (obligatorio): El nombre del jugador cuyas puntuaciones se desean obtener.

Parámetros opcionales para paginación:

- **page**: Número de página a recuperar. Por defecto es 1, no modificable desde la interfaz de usuario.
- **per_page**: Número de puntuaciones por página. Por defecto es 15, no modificable desde la interfaz de usuario.

Respuestas posibles:

- **Código 200**: Éxito. Se devuelve la lista de puntuaciones del jugador solicitado.
- **Código 400**: Error en los parámetros de paginación (por ejemplo, valores no enteros en page o per_page).
- **Código 403**: El jugador tiene su configuración de privacidad configurada como "privado" o "amigos" y no existe una relación de amistad aceptada con el jugador autenticado.
- **Código 404**: No se encontraron puntuaciones para el jugador especificado o el jugador no existe.

Respuesta de éxito:

json con los datos de la puntuación de el jugador solicitado filtrando por los que tengan privacidad publica y los que tengan privacidad amigos y sean amigos de usuario solicitantecop

DELETE /puntuaciones/limpiar

- Descripción: Elimina todas las puntuaciones de la base de datos. Requiere del Rol Admin y la única manera de acceder a este endpoint es mediante un curl que contenga el token de un usuario con el rol admin
- Respuesta:
 - Código 200 en caso de éxito.
 - Código 404 si el jugador no tiene puntuaciones o no existe.
 - Código 500 si ocurre un error durante la eliminación.
- Respuesta:
 - Se eliminan todas las puntuaciones del Leaderboard.

DELETE /puntuaciones/{nombre }

- Descripción: Elimina todas las puntuaciones de un jugador específico. Requiere del Rol *admin* y la única manera de acceder a este endpoint es mediante un curl que contenga el token de un usuario con el rol admin
- Parámetros: *`nombre`* (obligatorio): nombre del jugador cuyas puntuaciones se eliminarán.
- Respuesta:
 - Código 200 en caso de éxito.
 - Código 404 si el jugador no existe.
 - Código 500 si ocurre un error durante la eliminación.
- Respuesta en caso de éxito:
 - Se eliminan todas las puntuaciones asociadas al jugador.

DELETE /jugadores/limpiar

- Descripción: Elimina todos los jugadores de la base de datos. Requiere del Rol *admin* y la única manera de acceder a este endpoint es mediante un curl que contenga el token de un usuario con el rol *admin*
- Respuesta:
 - Código 200 en caso de éxito.
 - Código 404 si el jugador no tiene puntuaciones o no existe.
 - Código 500 si ocurre un error durante la eliminación.
- Respuesta:
 - Se eliminan todas las puntuaciones del Leaderboard.

C.1. Ejemplos de Endpoints de la API:

6.3. Dependencias Instaladas

Se utilizan varias librerías y extensiones para facilitar el desarrollo de la aplicación web en Flask. A continuación se detallan las principales dependencias:

- **Flask (==3.0.3):** Framework principal para la aplicación web, que proporciona las herramientas necesarias para desarrollar aplicaciones web robustas.
- **Flask-SQLAlchemy (==3.1.1):** Extensión que integra SQLAlchemy con Flask, facilitando la gestión de bases de datos en la aplicación.
- **Flask-Cors (==5.0.0):** Se utiliza para manejar las solicitudes CORS (Cross-Origin Resource Sharing), permitiendo que los clientes de diferentes orígenes interactúen con la API.
- **Flask-Mail (==0.10.0):** Extensión para enviar correos electrónicos desde la aplicación Flask. Se requiere configurar la conexión al servidor SMTP para enviar correos electrónicos.
- **Flask-Migrate (==4.0.7):** Extensión para manejar migraciones de bases de datos en Flask, utilizando Alembic para la gestión de cambios en el esquema de la base de datos.
- **requests (==2.31.0):** Biblioteca para realizar solicitudes HTTP. Es útil para interactuar con APIs externas.
- **psycopg2-binary (==2.9.9):** Driver para PostgreSQL que permite a la aplicación interactuar con bases de datos PostgreSQL.
- **PyJWT (==2.10.1):** Biblioteca para trabajar con JSON Web Tokens (JWT), utilizada para la generación y verificación de tokens seguros, como en el caso de restablecimiento de contraseñas.
- **Werkzeug (==3.0.4):** Biblioteca que proporciona utilidades para trabajar con aplicaciones web, como herramientas de seguridad y manejo de rutas.
- **greenlet (==3.1.1):** Biblioteca que proporciona soporte para la ejecución concurrente en aplicaciones Python, utilizada por SQLAlchemy.

- **SQLAlchemy (==2.0.34):** ORM (Object Relational Mapper) que facilita el trabajo con bases de datos relacionales al permitir el mapeo de objetos Python a tablas de bases de datos.

6.4. Servicio de Mail para la Recuperación de Contraseñas

En este sistema se ha integrado un servicio de correo electrónico utilizando Flask-Mail para permitir a los usuarios recuperar sus contraseñas. A continuación se describe la implementación:

Configuración de Flask-Mail

En la configuración de la aplicación Flask, se especifican los parámetros necesarios para el servidor SMTP que se utilizará para enviar los correos electrónicos. En este caso, se utiliza Gmail como servidor de correo:

```
app.config['MAIL_SERVER'] = 'smtp.gmail.com'
app.config['MAIL_PORT'] = 587
app.config['MAIL_USE_TLS'] = True
app.config['MAIL_USE_SSL'] = False
app.config['MAIL_USERNAME'] = 'deadvalleygame@gmail.com' # Correo electrónico de la aplicación
app.config['MAIL_PASSWORD'] = 'mwig lnax nfpw dwjg' # Contraseña de la cuenta de correo
app.config['MAIL_DEFAULT_SENDER'] = 'deadvalleygame@gmail.com' # Dirección de correo desde la que se
enviarán los mensajes
```

Ruta para Enviar Correo de Restablecimiento de Contraseña

En la ruta `/jugadores/send-reset-email`, el sistema recibe el correo del usuario, genera un token JWT para el restablecimiento de la contraseña y envía un correo con el enlace de restablecimiento.

```
@app.route('/jugadores/send-reset-email', methods=['POST'])
def send_reset_email():
    data = request.get_json()
    email = data.get('email')

    if not email:
        return jsonify({'error': 'El correo es obligatorio; Error 400'}), 400

    # Verificar si el correo está registrado
    usuario = Jugadores.query.filter_by(email=email).first()
    if not usuario:
        return jsonify({'error': 'El correo no está registrado; Error 400_2'}), 400

    # Crear un token JWT para el restablecimiento
    expiration_time = datetime.now(timezone.utc) + timedelta(hours=0.2)
    payload = {'email': email, 'exp': expiration_time}
    reset_token = jwt.encode(payload, app.config['SECRET_KEY'], algorithm='HS256')

    # Crear la URL del restablecimiento
    reset_url = f'http://localhost:5000/jugadores/reset-password/{reset_token}'

    # Crear el mensaje de correo
    subject = 'Restablece tu contraseña'
```

```

message = f'Haz clic en el siguiente enlace para restablecer tu contraseña:\n{reset_url}'
msg = Message(subject=subject, recipients=[email], body=message)

try:
    mail.send(msg) # Enviar el correo
    return jsonify({'message': 'Correo enviado exitosamente'}), 200
except Exception as e:
    return jsonify({'error': f'Error al enviar el correo: {str(e)}'}), 500

```

Ruta para Restablecer la Contraseña

La ruta `/jugadores/reset-password/<reset_token>` se encarga de procesar el token de restablecimiento enviado por correo. Si el token es válido, se solicita al usuario que ingrese su nueva contraseña. Si el token ha expirado o es inválido, se devuelve un error.

```

@app.route('/jugadores/reset-password/<reset_token>', methods=['GET', 'POST'])
def reset_password(reset_token):
    try:
        payload = jwt.decode(reset_token, app.config['SECRET_KEY'], algorithms=['HS256'])
        email = payload.get('email')
        usuario = Jugadores.query.filter_by(email=email).first()

        if not usuario:
            return jsonify({'error': 'El usuario no existe; Error 400_3'}), 400

        # Verificar si el token ha expirado
        if payload.get('exp') < datetime.now(timezone.utc).timestamp():
            return jsonify({'error': 'El token ha expirado; Error 511'}), 511

        if request.method == 'POST':
            new_password = request.form.get('password')
            repeat_password = request.form.get('repeatpassword')

            if not new_password or not repeat_password:
                return jsonify({'error': 'Ambos campos de contraseña son obligatorios; Error 400_5'}), 400

            if new_password != repeat_password:
                return jsonify({'error': 'Las contraseñas no coinciden; Error 400_6'}), 400

            usuario.password = generate_password_hash(new_password)
            db.session.commit()

            return render_template_string("""
            <h1>Contraseña restablecida exitosamente</h1>
            <p>Ahora puedes iniciar sesión con tu nueva contraseña.</p>
            """)
        except jwt.ExpiredSignatureError:
            return jsonify({'error': 'El token ha expirado; Error 400_7'}), 400
        except jwt.InvalidTokenError:
            return jsonify({'error': 'Token inválido; Error 400_8'}), 400

```

Descripción de la Flujo de Trabajo

- **Paso 1:** El usuario ingresa su correo electrónico y solicita un correo para restablecer su contraseña.

- **Paso 2:** El sistema genera un token JWT que contiene la dirección de correo electrónico y la fecha de expiración.
- **Paso 3:** Se envía un correo electrónico al usuario con un enlace que contiene el token.
- **Paso 4:** El usuario accede al enlace, donde se le permite ingresar una nueva contraseña.
- **Paso 5:** El sistema valida el token y si es válido, guarda la nueva contraseña en la base de datos.

Manejo de Errores

- Si el correo no está registrado en la base de datos, se devuelve un error 400.
- Si el token ha expirado, se devuelve un error 511.
- Si el token es inválido, se devuelve un error 400_8.
- Si ocurre un error en el proceso de envío del correo, se captura la excepción y se devuelve un error 500.

E.1. Implementación del Servicio de Correo Electrónico

6.5. Explicación JWT para Autenticación

La implementación de la autenticación basada en JSON Web Tokens (JWT) en la API tiene como objetivo garantizar la seguridad en las operaciones realizadas por los usuarios. Este sistema permite autenticar usuarios y restringir el acceso a recursos según permisos. A continuación, se detalla el proceso de implementación:

1. Generación del Token JWT Al registrar o autenticar a un usuario, se genera un token JWT firmado con una clave secreta almacenada de manera segura en la configuración de la aplicación. Se utiliza la librería jwt para crear el token. El token incluye un payload que contiene información relevante del usuario, como su identificador único (id) y un tiempo de expiración (exp). Ejemplo del proceso:

```
expiration_time = datetime.now(timezone.utc) + timedelta(hours=1)
payload = {
    'id': usuario.id,
    'exp': expiration_time
}
token = jwt.encode(payload, Config.SECRET_KEY, algorithm='HS256')
```

2. Protección de Rutas con Decoradores Las rutas que requieren autenticación están protegidas mediante decoradores personalizados que verifican la validez del token JWT. Se implementaron dos decoradores:
 - **authenticate_token:** Verifica la existencia y validez del token, decodificando su contenido para recuperar la información del usuario. Además, comprueba si el usuario asociado al token existe en la base de datos.
 - **authenticate_token2:** Extiende la funcionalidad del anterior, verificando adicionalmente si el usuario tiene un rol específico requerido para acceder a la ruta.

Ejemplo de uso:

```
@app.route('/ruta-prottegida', methods=['GET'])
```

```
@authenticate_token
def ruta_protegida():
    return jsonify({'message': 'Acceso permitido'})
```

3. Validación del Token Al acceder a rutas protegidas, se espera que el cliente proporcione el token en el encabezado Authorization con el formato **Bearer <token>**. El decorador verifica:

- Si el token está presente.
- Si el token es válido y no ha expirado.
- Si el usuario asociado al token existe en la base de datos.

En caso de errores (token ausente, expirado o inválido), se devuelven respuestas apropiadas con códigos de estado como 403, 401 o 404.

4. Uso en Funcionalidades Clave

- **Registro de Usuarios:** Se utiliza para registrar jugadores con contraseñas encriptadas utilizando `generate_password_hash` de `werkzeug.security`. El token no se genera en este paso, pero puede añadirse como funcionalidad posterior.
- **Restablecimiento de Contraseña:**
 - Se genera un token JWT con tiempo de expiración reducido, que se envía al correo electrónico del usuario.
 - El usuario puede usar este token para acceder a un formulario y restablecer su contraseña.
- **Restricción Basada en Roles:**
 - En rutas sensibles, como aquellas accesibles solo por administradores, se utiliza el decorador `authenticate_token2` para validar roles de usuario.

5. Seguridad

- Se utiliza el algoritmo HS256 para firmar los tokens.
- Los tokens tienen un tiempo de expiración definido, reduciendo riesgos asociados con tokens comprometidos.
- Las contraseñas de los usuarios están encriptadas antes de almacenarse en la base de datos.

Esta implementación permite un control robusto sobre la autenticación y autorización de usuarios, asegurando que solo los usuarios autorizados puedan acceder a recursos sensibles y funcionalidades específicas.

B.3. Detalles de Implementación de JWT para Autenticación

6.6. Configuración de CORS

En el contexto de la API desarrollada con Flask, la configuración de **Cross-Origin Resource Sharing (CORS)** permite que los recursos del servidor sean accesibles desde dominios externos. CORS es un mecanismo de seguridad que ayuda a controlar qué orígenes (dominios) pueden interactuar con la API, lo que es especialmente útil cuando se desarrollan aplicaciones frontend que están separadas del servidor backend.

Descripción de la Configuración:

La configuración de CORS en este proyecto se maneja a través del paquete `flask_cors` que proporciona una forma sencilla de gestionar las políticas de CORS para aplicaciones Flask.

1. Instalación de `flask_cors`:

Para usar CORS en la API, primero se debe instalar el paquete `flask_cors`. Se puede hacer mediante `pip`:

```
pip install flask-cors
```

2. Configuración en `app.py`:

En el archivo principal de la aplicación (`app.py`), después de importar la extensión CORS, se configura con la siguiente línea:

```
CORS(app, origins=Config.CORS_ALLOWED_ORIGINS)
```

Esto permite que la API acepte solicitudes CORS solo desde los orígenes especificados en la lista `CORS_ALLOWED_ORIGINS` de la configuración.

3. Configurar Orígenes Permitidos:

En el archivo de configuración (`config.py`), el parámetro `CORS_ALLOWED_ORIGINS` se define como una lista de URLs permitidas. Esto restringe las solicitudes CORS a los dominios especificados. Si no se incluyen dominios, se bloquean todas las solicitudes desde otros orígenes.

```
CORS_ALLOWED_ORIGINS = [  
    'http://localhost:3000', # Dominio permitido para desarrollo local  
    'https://misitio.com',  # Dominio de producción o frontend autorizado  
]
```

4. Configuración de Encabezados Permitidos:

El parámetro `CORS_HEADERS` se utiliza para permitir ciertos encabezados en las solicitudes CORS. En este caso, se han permitido los encabezados `Content-Type` y `Authorization` para que las solicitudes puedan enviar contenido y tokens de autenticación de manera segura.

```
CORS_HEADERS = 'Content-Type', 'Authorization'
```

5. Soporte para Credenciales:

El parámetro `CORS_SUPPORTS_CREDENTIALS` se ha configurado como `True` para permitir que las solicitudes incluyan credenciales (por ejemplo, cookies o encabezados de autenticación). Esto es importante cuando la API necesita interactuar con aplicaciones que usan autenticación basada en sesiones o tokens.

```
CORS_SUPPORTS_CREDENTIALS = True
```

Ejemplo de Uso:

Imaginemos que tienes un cliente frontend en `http://localhost:3000` y deseas que este cliente pueda interactuar con la API ubicada en tu servidor Flask (por ejemplo, en `http://localhost:5000`). Gracias a la configuración de CORS, solo se permitirá que el frontend haga solicitudes a la API si el origen de la solicitud es `http://localhost:3000` o algún otro dominio autorizado.

Ventajas de la Configuración:

1. **Seguridad:** Al restringir los orígenes que pueden acceder a la API, se protege contra solicitudes no deseadas de dominios no autorizados.
2. **Facilidad de desarrollo:** Durante el desarrollo, puedes permitir orígenes locales como `localhost` para pruebas sin preocuparte por las restricciones de CORS.
3. **Flexibilidad:** Puedes actualizar fácilmente la lista de orígenes permitidos en la configuración según las necesidades del proyecto (por ejemplo, agregar dominios de producción).

Consideraciones Adicionales:

- **Desarrollo vs. Producción:** Durante el desarrollo local, se puede permitir `localhost`, pero en un entorno de producción se recomienda restringir las solicitudes a dominios específicos para mayor seguridad.
- **Manejo de Errores CORS:** Si una solicitud de origen no permitido intenta acceder a la API, el navegador bloqueará la solicitud, y el servidor debería responder con un mensaje adecuado (por ejemplo, error 403).

Ejemplo de Configuración:

```
# Configuración de CORS en `config.py`
```

```
class Config:
    SQLALCHEMY_DATABASE_URI = 'postgresql://client:sesamoPass1?@localhost:5432/game'
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    CORS_SUPPORTS_CREDENTIALS = True
    SECRET_KEY = '6Nj3G€%l&k!YgFs8P?fds'
    CORS_HEADERS = 'Content-Type', 'Authorization'
    CORS_ALLOWED_ORIGINS = [
        'http://localhost:3000', # Origen permitido durante desarrollo
        'https://misitio.com',  # Origen permitido en producción
    ]
```

Con esta configuración, la API estará protegida contra solicitudes no autorizadas mientras mantiene la flexibilidad necesaria para trabajar en un entorno de desarrollo y producción, Actualmente no es necesario obtener datos de ninguna dirección de diferente dominio pero en caso de que sea necesario, esta implementación nos proporciona una capa de seguridad.

E.2. Configuración de CORS

6.7. Generacion Informes

La generacion de informes es llevada a cabo el admin de la base de datos mediante un programa alojado en la carpeta informes, al ejecutarlo se generan los siguientes informes:

Informe de jugadores					
jueves 02 enero 2025					
Jugador ID	Nombre	Email	Telefono	Nivel de privacidad	Valor puntuacion
5	jugador2	jugador2@gmail.com	638703511	amigos	99
4	jugador1	jugador1@gmail.com	638703516	amigos	99

Informe de puntuaciones		
jueves 02 enero 2025		
Puntuacion ID	Nombre	Valor puntuacion
2	jugador2	99
3	jugador1	99

código:

```
public class JasperReportExecutor {  
  
    public static void main(String[] args) throws JRException {
```

```

Connection conn = null;

try {
    // Configurar la conexión a la base de datos
    String dbUrl = "jdbc:postgresql://localhost:5432/game";
    String dbUser = "admin";
    String dbPassword = "P@ssw0rd1?";

    // Intentar obtener la conexión a la base de datos
    conn = DriverManager.getConnection(dbUrl, dbUser, dbPassword);

    // Verificar si la conexión fue exitosa
    if (conn != null && conn.isValid(5)) {
        System.out.println("Conexión a la base de datos establecida correctamente.");

        // Ruta al archivo .jasper (compilado desde Jaspersoft Studio)
        String reportPath =
JasperReportExecutor.class.getClassLoader().getResource("informePuntuaciones.jasper").getPath();
        //String absoluteReportPath = "C:\\Users\\User\\Desktop\\webapp_puntuaciones\\
informes\\informe.jasper";

        // Parámetros del informe (si fuesen necesarios)
        //Map<String, Object> parameters = new HashMap<>();
        //parameters.put("param1", "valor1"); // Ejemplo de un parámetro

        // Llenar el informe con datos
        JasperPrint jasperPrint = JasperFillManager.fillReport(reportPath, null, conn); // Aquí
añadirías los parámetros si los hubieras

        // Obtener la ruta base del directorio del proyecto (suponiendo que sea el directorio raíz
del proyecto)
        String basePath = System.getProperty("user.dir"); // Devuelve el directorio de trabajo
actual

        // Construir la ruta relativa a 'webapp_puntuaciones\\informes\\informe.pdf'
        Path outputPath = Paths.get(basePath, "..", "informePuntuaciones.pdf");

        // Convertir la ruta a una cadena para poder usarla
        String pdfOutputPath = outputPath.toString();

        // Exportar el informe a PDF
        JasperExportManager.exportReportToPdfFile(jasperPrint, pdfOutputPath);
        System.out.println("Informe generado en: " + pdfOutputPath);

        // Ruta al archivo .jasper (compilado desde Jaspersoft Studio)
        String reportPath2 =
JasperReportExecutor.class.getClassLoader().getResource("informeJugadores.jasper").getPath();

```

```

        // Llenar el informe con datos
        JasperPrint jasperPrint2 = JasperFillManager.fillReport(reportPath2, null, conn); // Aquí
añadirías los parámetros si los hubieras

        // Construir la ruta relativa a 'webapp_puntuaciones\\informes\\informe.pdf'
        Path outputPath2 = Paths.get(basePath, "..", "informeJugadores.pdf");

        // Convertir la ruta a una cadena para poder usarla
        String pdfOutputPath2 = outputPath2.toString();

        // Exportar el informe a PDF
        JasperExportManager.exportReportToPdfFile(jasperPrint2, pdfOutputPath2);
        System.out.println("Informe generado en: " + pdfOutputPath2);

    } else {
        System.out.println("Conexion fallida.");
    }

} catch (SQLException a) {
    System.out.println("Error al conectar a la base de datos");
    a.printStackTrace();
} finally {
    try {
        // Cerrar la conexión a la base de datos si se abrió
        if (conn != null && !conn.isClosed()) {
            conn.close();
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}
}
}
}

```

E.3. Generación de informes

6.8. Frontend (Cliente Unity)

- **Motor de Juego:** Unity
- **Lenguaje de Programación:** C#
- **Manejo de JSON:** SimpleJSON

- **Interfaz Gráfica del Usuario (GUI):**
- **Pantalla de Registro:** Permite a los usuarios registrarse en el sistema proporcionando su información personal.
- **Pantalla de Login:** Permite a los usuarios iniciar sesión en el sistema e incluye una opción para recuperar la contraseña en caso de olvido.

- **Pantalla de Leaderboard:** Muestra las puntuaciones de los jugadores de manera ordenada. Incluye opciones de filtrado, permitiendo a los usuarios buscar por nombre de jugador y ordenar las puntuaciones de mayor a menor o viceversa.
- **Pantalla de Friendsboard:** Muestra los amigos del jugador permitiendo a los usuarios enviar una solicitud de amistad a un jugador por su nombre y ver la lista de solicitudes pendientes por responder, así como aceptar o rechazar las solicitudes.
- **Perfil del Jugador:** Permite a los jugadores visualizar y editar sus datos personales, así como su nivel de privacidad. Los jugadores pueden elegir el nivel de privacidad de su cuenta (público, privado o amigos). Este ajuste impacta en la visibilidad de la información del jugador, como las puntuaciones
- **Opciones:** Permite a los jugadores ajustar configuraciones del juego como el volumen, activar o desactivar el sonido, y modificar las opciones gráficas.
- **Notificaciones:** Se han añadido alertas para notificar al usuario de los eventos en la aplicación (registro exitoso, login exitoso, datos del perfil actualizados correctamente, etc)



Componentes Principales:

1. Interacción con la API:

- **Envío de Puntuaciones:** Funciones que permiten al juego enviar las puntuaciones al servidor al finalizar una partida.

```
public void EnviarPuntuacion(int valor)
{
    // Obtener el nombre de usuario desde el AuthController
    string nombreUsuario = AuthController.Instance.loginUsernameField.text; //
    // Obtiene el nombre desde el campo de login
```



```

if (!string.IsNullOrEmpty(nombreUsuario))
{
    // Imprimir en la consola de Unity el nombre del usuario
    Debug.Log("Nombre de usuario obtenido: " + nombreUsuario);
}
else
{
    Debug.LogError("No se ha encontrado el nombre de usuario.");
    return;
}
StartCoroutine(PostPuntuacion(nombreUsuario, valor));
}

private IEnumerator PostPuntuacion(string nombre, int valor)
{
    string url = apiUrl; // Asegúrate de que `apiUrl` esté correctamente configurada

    // Crear el objeto JSON utilizando SimpleJSON
    var puntuacionData = new JSONObject();
    puntuacionData.Add("nombre", nombre);
    puntuacionData.Add("valor", valor);

    string jsonData = puntuacionData.ToString(); // Convertir el objeto JSON a string

    string authToken = AuthController.GetAuthToken();

    if (string.IsNullOrEmpty(authToken))
    {
        Debug.LogError("No token encontrado. El usuario no está autenticado.");
        yield break;
    }

    // Usar UnityWebRequest para un POST con JSON
    using (UnityWebRequest www = new UnityWebRequest(url, "POST"))
    {
        // Subir los datos JSON como cuerpo de la solicitud
        www.uploadHandler = new
UploadHandlerRaw(System.Text.Encoding.UTF8.GetBytes(jsonData));
        www.downloadHandler = new DownloadHandlerBuffer();

        // Establecer los encabezados correctamente
        www.SetRequestHeader("Content-Type", "application/json"); // Necesario para
que el servidor reconozca el JSON
        www.SetRequestHeader("Authorization", "Bearer " + authToken); // Token JWT
para autorización

        yield return www.SendWebRequest();

        if (www.result == UnityWebRequest.Result.ConnectionError || www.result ==
UnityWebRequest.Result.ProtocolError)
        {
            Debug.LogError(www.error);
            // Manejo de las respuestas HTTP según el código de estado
            switch (www.responseCode)
            {
                case 400: // Solicitud incorrecta
                    Debug.LogError("Error 400: Solicitud incorrecta. Verifica los
datos enviados.");
                    break;

                case 422: // Solicitud incorrecta
                    Debug.LogError("Error 422: La nueva puntuación debe ser mayor que
la actual.");
            }
        }
    }
}

```

```

        break;

        case 409: // Conflicto: el jugador ya existe
            // En caso de que exista ya la puntuación tratamos de actualizarla
            Debug.Log("Actualizando puntuación");
            //StartCoroutine(ActualizarPuntuacion(nombre, valor));
            Debug.Log("Error 409: Conflicto. El jugador ya existe. Tratamos de
actualizar la puntuación");
            break;

        case 500: // Error en el servidor
            Debug.LogError("Error 500: Error en el servidor. Intenta más
tarde.");
            break;

        default:
            // Manejo de otros códigos de estado no esperados
            Debug.LogError($"Error {www.responseCode}: {www.error}. Respuesta
del servidor: {www.downloadHandler.text}");
            break;
    }
}
else
{
    // Manejo de las respuestas HTTP según el código de estado
    switch (www.responseCode)
    {
        case 201:
        case 200:// Creado correctamente
            Debug.Log("Puntuación enviada correctamente.");

            // Deserializar la respuesta si es un JSON
            string responseText = www.downloadHandler.text;
            PuntuacionResponse2 response =
JsonUtility.FromJson<PuntuacionResponse2>(responseText);

            // Mostrar los detalles de la respuesta
            Debug.Log($"Puntuación agregada con ID: {response.id}, Valor:
{response.valor}");
            Debug.Log($"Jugador: {response.jugador.nombre} (ID:
{response.jugador.id})");
            break;

        default:
            // Manejo de otros códigos de estado no esperados
            Debug.LogError($"Error {www.responseCode}: {www.error}. Respuesta
del servidor: {www.downloadHandler.text}");
            break;
    }
}
}
}

```

- **Recuperación de Puntuaciones:** Funciones para obtener y mostrar las puntuaciones desde la API. Se recuperan las puntuaciones teniendo en cuenta los niveles de privacidad del jugador. Si una puntuación tiene un nivel de privacidad como "amigos", solo los amigos del jugador pueden verla; si está configurada como "público", cualquier jugador podrá verla; si está configurada como "privado", solo el jugador dueño de la puntuación podrá acceder a ella.

```

// Método para botón de ordenar puntuaciones de manera ascendente
public void OnOrderAscendenteButtonClick()
{
    GetPuntuacionesOrdenadas("asc", 1);
    usadoOrderAsc = true;
    UpdatePaginationUI();
}

// Método para botón de ordenar puntuaciones de manera descendente
public void OnOrderDescendenteButtonClick()
{
    GetPuntuacionesOrdenadas("desc", 1);
    usadoOrderAsc = false;
    UpdatePaginationUI();
}

// Método para obtener puntuaciones ordenadas por valor (ascendente o descendente) y paginadas
public void GetPuntuacionesOrdenadas(string orden, int page)
{
    leaderboardUI.ClearLeaderboardUI();
    StartCoroutine(GetPuntuacionesOrdenadasPorValorYPagina(orden, page));
}
private IEnumerator GetPuntuacionesOrdenadasPorValorYPagina(string orden, int page)
{
    //Debug.Log("get");
    //Debug.LogError(orden);
    //Debug.LogError(page);
    // No pasamos los parámetros de página si no son necesarios, ya que tienen valores
por defecto en el servidor
    string authToken = AuthController.GetAuthToken();
    if (string.IsNullOrEmpty(authToken))
    {
        Debug.LogError("No token encontrado. El usuario no está autenticado.");
        yield break;
    }

    string url = apiUrl + $"?order_by=valor&order={orden}&page={page}";

    using (UnityWebRequest www = UnityWebRequest.Get(url))
    {
        www.SetRequestHeader("Authorization", "Bearer " + authToken); // Añadir token
a la cabecera
        yield return www.SendWebRequest();

        if (www.result == UnityWebRequest.Result.ConnectionError || www.result ==
UnityWebRequest.Result.ProtocolError)
        {
            Debug.LogError(www.error);
        }
        else
        {
            string jsonResult = www.downloadHandler.text;

            // Usamos SimpleJSON para parsear el JSON
            JSONNode puntuacionesJson = JSON.Parse(jsonResult);

            // Validar que el JSON recibido contiene datos válidos
            if (puntuacionesJson == null || puntuacionesJson["puntuaciones"] == null)

```

```

        {
            Debug.LogError("No se recibieron puntuaciones válidas del servidor.");
            yield break; // Salir si el JSON no es válido
        }

        // Mostramos las puntuaciones ordenadas
        // DisplayPuntuaciones(puntuacionesJson["puntuaciones"]);
        Debug.Log(puntuacionesJson.ToString()); // Esto te dará una representación
completa del JSON

        if (leaderboardUI != null)
        {
            // Actualizamos la UI con las puntuaciones obtenidas
            leaderboardUI.UpdateLeaderboardUI(puntuacionesJson);
        } else {
            Debug.LogError("El objeto 'leaderboardUI' no está asignado en el
inspector.");
        }

        // Actualizamos el número total de páginas desde la respuesta (si el
servidor lo proporciona)
        if (puntuacionesJson["total_pages"] != null)
        {
            totalPages = puntuacionesJson["total_pages"].AsInt;
        }

        //Debug.Log(totalPages);
    }
}

```

```

// Método para botón de mostrar puntuaciones por usuario
public void OnShowPuntuacionesUsuarioButtonClick()
{
    // Asegúrate de que el TMP_InputField 'nombreOrder' tiene un valor válido
    if (nombreOrder != null)
    {
        // Tomamos el texto del TMP_InputField y lo usamos directamente como nombre
string nombreUsuario = nombreOrder.text.Trim(); // Eliminamos espacios en
blanco

        if (!string.IsNullOrEmpty(nombreUsuario)) // Verificamos que no esté vacío
        {
            GetPuntuacionesPorUsuario(nombreUsuario);
        }
        else
        {
            Debug.LogWarning("El campo 'nombreOrder' no contiene un nombre de usuario
válido.");
        }
    }
    else
    {
        Debug.LogError("El objeto 'nombreOrder' no está asignado en el Inspector.");
    }
}

```

```

}

// Método para obtener puntuaciones de un usuario específico
public void GetPuntuacionesPorUsuario(string nombreUsuario)
{
    // Limpiar la UI antes de hacer la solicitud
    leaderboardUI.ClearLeaderboardUI();
    StartCoroutine(GetPuntuacionesUsuario(nombreUsuario));
}
private IEnumerator GetPuntuacionesUsuario(string nombreUsuario)
{
    string authToken = AuthController.GetAuthToken();
    if (string.IsNullOrEmpty(authToken))
    {
        Debug.LogError("No token encontrado. El usuario no está autenticado.");
        yield break;
    }

    string url = apiUrl + $"/{nombreUsuario}"; // Cambiamos el endpoint para usar
    nombre

    using (UnityWebRequest www = UnityWebRequest.Get(url))
    {
        www.SetRequestHeader("Authorization", "Bearer " + authToken); // Añadir token
        a la cabecera
        yield return www.SendWebRequest();

        if (www.result == UnityWebRequest.Result.ConnectionError || www.result ==
        UnityWebRequest.Result.ProtocolError)
        {
            Debug.LogError(www.error);
        }
        else
        {
            string jsonResult = www.downloadHandler.text;

            // Usamos SimpleJSON para parsear el JSON
            JSONNode puntuacionesJson = JSON.Parse(jsonResult);

            // Validar que el JSON recibido contiene datos válidos
            if (puntuacionesJson == null || puntuacionesJson["puntuaciones"] == null)
            {
                Debug.LogError("No se recibieron puntuaciones válidas del servidor.");
                Debug.Log(puntuacionesJson.ToString()); // Esto te dará una
                representación completa del JSON
                yield break; // Salir si el JSON no es válido
            }

            // Mostramos las puntuaciones del usuario
            DisplayPuntuaciones(puntuacionesJson["puntuaciones"]);

            if (leaderboardUI != null)
            {
                Debug.Log(puntuacionesJson.ToString()); // Esto te dará una
                representación completa del JSON

                // Actualizamos la UI con las puntuaciones obtenidas
                leaderboardUI.UpdateLeaderboardUI(puntuacionesJson);
            }
            else
            {
                Debug.LogError("El objeto 'leaderboardUI' no está asignado en el
                inspector.");
            }
        }
    }
}

```

```

    }
}

// Método para cambiar la página cuando sea necesario
public void LoadPage(int page)
{
    leaderboardUI.ClearLeaderboardUI();
    // Pasa la página solo cuando se necesita cambiar
    //Debug.Log("load page" + page );
    if (usadoOrderAsc)
    {
        GetPuntuacionesOrdenadas("asc", page);
    }else {
        GetPuntuacionesOrdenadas("desc", page);
    }
}

// Método para actualizar la UI de paginación
private void UpdatePaginationUI()
{
    if (currentPageText != null)
    {
        currentPageText.text = "Página: " + currentPage;
    }

    previousPageButton.interactable = currentPage > 1;
    nextPageButton.interactable = currentPage < totalPages;
}

// Método para el botón de "Página Anterior"
public void OnPreviousPageButtonClick()
{
    if (currentPage > 1)
    {
        currentPage--;
        LoadPage(currentPage);
        UpdatePaginationUI();
    }
}

// Método para el botón de "Siguiente Página"
public void OnNextPageButtonClick()
{
    Debug.Log("next button");
    if (currentPage < totalPages)
    {
        currentPage++;
        LoadPage(currentPage);
        UpdatePaginationUI();
    }
}

```

- **Gestión del Perfil de Usuario:**

Funciones que permiten al jugador actualizar su perfil, incluyendo su información personal y las preferencias de privacidad.

El jugador puede modificar el nivel de privacidad de su cuenta, seleccionando entre "público", "privado" o "amigos".

Cuando el jugador cambia el nivel de privacidad, se actualizan las configuraciones de visibilidad de sus puntuaciones, solicitudes de amistad y otros datos relacionados.

```
private string profileEndp = "http://localhost:5000/jugadores/perfil";
// Método público para cargar el perfil del usuario
public void CargarPerfilUsuario()
{
    StartCoroutine(GetPerfil());
}

// Corutina para obtener los datos del perfil
private IEnumerator GetPerfil()
{
    string authToken = AuthController.GetAuthToken(); // Recupera el token de
autenticación
    Debug.Log("Token de autenticación: " + authToken);

    if (string.IsNullOrEmpty(authToken))
    {
        Debug.LogError("No token encontrado. El usuario no está autenticado.");
        yield break;
    }

    using (UnityWebRequest request = UnityWebRequest.Get(profileEndp))
    {
        Debug.Log("Token: " + authToken);

        // Agregar el token JWT en la cabecera Authorization
        request.SetRequestHeader("Authorization", "Bearer " + authToken);

        // Enviar la solicitud
        yield return request.SendWebRequest();

        if (request.result == UnityWebRequest.Result.ConnectionError ||
request.result == UnityWebRequest.Result.ProtocolError)
        {
            Debug.LogError($"Error en la solicitud: {request.error}");
            yield break;
        }

        // Procesar la respuesta JSON
        string jsonResponse = request.downloadHandler.text;
        try
        {
            JSONNode perfilData = JSON.Parse(jsonResponse);

            // Verificar si el perfil existe
            if (perfilData.HasKey("error"))
            {
                Debug.LogError("Usuario no encontrado: " + perfilData["error"]);
                yield break;
            }

            // Actualizar los campos de texto con los datos del perfil
```

```

        idText.text = perfilData["id"];
        nombreInput.text = perfilData["nombre"];
        emailInput.text = perfilData["email"];
        telefonoInput.text = perfilData["numtelefono"];

        // Obtener el ID del jugador para la solicitud de privacidad
        int jugadorId = perfilData["id"].AsInt;

        // Llamada para obtener el nivel de privacidad del jugador
        StartCoroutine(GetPrivacidad(jugadorId));
    }
    catch (System.Exception ex)
    {
        Debug.LogError("Error al procesar la respuesta JSON: " + ex.Message);
    }
}

private string privacyEndp = "http://localhost:5000/privacidad";

private IEnumerator GetPrivacidad(int jugadorId)
{
    string authToken = AuthController.GetAuthToken(); // Recupera el token de
    autenticación

    using (UnityWebRequest request = UnityWebRequest.Get(privacyEndp +
        $"/{jugadorId}"))
    {
        Debug.Log("Token: " + authToken);

        // Agregar el token JWT en la cabecera Authorization
        request.SetRequestHeader("Authorization", "Bearer " + authToken);

        // Enviar la solicitud
        yield return request.SendWebRequest();

        if (request.result == UnityWebRequest.Result.ConnectionError ||
            request.result == UnityWebRequest.Result.ProtocolError)
        {
            Debug.LogError($"Error en la solicitud de privacidad:
            {request.error}");
            yield break;
        }

        // Procesar la respuesta JSON de privacidad
        string jsonResponse = request.downloadHandler.text;
        try
        {
            JSONNode privacidadData = JSON.Parse(jsonResponse);

            // Verificar si el nivel de privacidad existe
            if (privacidadData.HasKey("message"))
            {
                Debug.LogError("Privacidad no encontrada: " +
                    privacidadData["message"]);
                yield break;
            }

            // Obtener el nivel de privacidad
            string privacidad = privacidadData["nivel_de_privacidad"];

            // Actualizar el valor del Dropdown según la privacidad almacenada

```



```

        PrivacyLevel privacy =
(PrivacyLevel)System.Enum.Parse(typeof(PrivacyLevel), privacidad);
        privacyDropdown.value = (int)privacy;
    }
    catch (System.Exception ex)
    {
        Debug.LogError("Error al procesar la respuesta JSON de privacidad: " +
ex.Message);
    }
}

// Método público para actualizar el perfil del usuario
public void ActualizarPerfilUsuario()
{
    StartCoroutine(PutPerfil());
}

// Corutina para actualizar los datos del perfil
private IEnumerator PutPerfil()
{
    string authToken = AuthController.GetAuthToken();
    Debug.Log("Token de autenticación: " + authToken);

    if (string.IsNullOrEmpty(authToken))
    {
        Debug.LogError("No token encontrado. El usuario no está autenticado.");
        yield break;
    }

    // Validar que los campos no estén vacíos
    if (string.IsNullOrEmpty(nombreInput.text) ||
string.IsNullOrEmpty(emailInput.text) || string.IsNullOrEmpty(telefonoInput.text))
    {
        Debug.LogError("Uno o más campos están vacíos. No se puede enviar la
solicitud.");
        yield break;
    }

    // Crear el JSON con los datos actualizados
    JSONNode perfilData = new JSONObject();
    perfilData["nombre"] = nombreInput.text;
    perfilData["email"] = emailInput.text;
    perfilData["numtelefono"] = telefonoInput.text;

    // Obtener el valor seleccionado del Dropdown para privacidad
    PrivacyLevel selectedPrivacy = (PrivacyLevel)privacyDropdown.value;
    perfilData["privacidad"] = selectedPrivacy.ToString();

    // Log para depuración
    Debug.Log("Datos enviados al servidor: " + perfilData.ToString());

    using (UnityWebRequest request = UnityWebRequest.Put(profileEndp,
perfilData.ToString()))
    {
        request.SetRequestHeader("Authorization", "Bearer " + authToken);
        request.SetRequestHeader("Content-Type", "application/json");

        yield return request.SendWebRequest();
    }
}

```

```

        if (request.result == UnityWebRequest.Result.ConnectionError ||
request.result == UnityWebRequest.Result.ProtocolError)
        {
            Debug.LogError($"Error al actualizar perfil: {request.error}");
            yield break;
        }

        // Si todo ha salido bien, mostrar un mensaje
        if (request.result == UnityWebRequest.Result.Success)
        {
            Debug.Log("Perfil actualizado exitosamente: " +
request.downloadHandler.text);

            // Ahora actualizamos la privacidad
            yield return StartCoroutine(ActualizarPrivacidad(selectedPrivacy));
        }
        else
        {
            // Si la respuesta del servidor es otro código, logueamos la respuesta
            Debug.LogError("Error inesperado al actualizar perfil: " +
request.downloadHandler.text);
        }
    }

    // Corutina para actualizar la privacidad del perfil
    private IEnumerator ActualizarPrivacidad(PrivacyLevel selectedPrivacy)
    {
        string authToken = AuthController.GetAuthToken(); // Recupera el token de
autenticación
        int jugadorId = int.Parse(idText.text); // Suponiendo que el 'idText' contiene
el ID del jugador

        if (string.IsNullOrEmpty(authToken))
        {
            Debug.LogError("No token encontrado. El usuario no está autenticado.");
            yield break;
        }

        // Crear el JSON con el nivel de privacidad
        JSONNode privacidadData = new JSONObject();
        privacidadData["nivel_de_privacidad"] = selectedPrivacy.ToString();

        // Log para depuración
        Debug.Log("Datos de privacidad enviados al servidor: " +
privacidadData.ToString());

        // Realizar la solicitud PUT para actualizar la privacidad
        using (UnityWebRequest request = UnityWebRequest.Put(privacyEndp +
$"/{jugadorId}", privacidadData.ToString()))
        {
            request.SetRequestHeader("Authorization", "Bearer " + authToken);
            request.SetRequestHeader("Content-Type", "application/json");

            yield return request.SendWebRequest();

            if (request.result == UnityWebRequest.Result.ConnectionError ||
request.result == UnityWebRequest.Result.ProtocolError)
            {
                Debug.LogError($"Error al actualizar privacidad: {request.error}");
                yield break;
            }
        }
    }

```

```

        // Verificar si la actualización fue exitosa
        if (request.result == UnityWebRequest.Result.Success)
        {
            Debug.Log("Privacidad actualizada exitosamente: " +
request.downloadHandler.text);
        }
        else
        {
            Debug.LogError("Error al actualizar privacidad: " +
request.downloadHandler.text);
        }
    }
}

```

```

// Definimos el enum para la privacidad
public enum PrivacyLevel
{
    publico,
    privado,
    amigos
}

```

- **Gestión de Amigos:**

Funciones para enviar y recibir solicitudes de amistad.

Los jugadores pueden enviar solicitudes de amistad a otros jugadores buscando por nombre.

Se pueden ver las solicitudes de amistad pendientes y aceptarlas o rechazarlas.

Cuando dos jugadores se convierten en amigos, pueden ver las puntuaciones del otro siempre que el otro tenga su privacidad configurada como *amigos* o *public*.

La relación de amistad también impacta en las opciones de privacidad, permitiendo que los jugadores ajusten las configuraciones de visibilidad basadas en si tienen o no a una persona como amigo.

```

private void Start()
{
    ObtenerAmigosAsync();
}

void OnLoad()
{
    // Asegurarse de que la referencia al LeaderBoardUI esté configurada
    if (PanelFriendsUI == null)
    {
        PanelFriendsUI = FindObjectOfType<PanelFriendsUI>();
    }
}

```

```

// Método público para enviar una solicitud de amistad
public void EnviarSolicitudPublic()
{
    StartCoroutine(ObtenerReceptorIdYEnviarSolicitud());
}

// Método para obtener el ID del receptor a partir del nombre y luego enviar la solicitud
private IEnumerator ObtenerReceptorIdYEnviarSolicitud()
{
    string authToken = AuthController.GetAuthToken();
    if (string.IsNullOrEmpty(authToken))
    {
        Debug.LogError("No se encontró token. El usuario no está autenticado.");
        yield break;
    }

    // Obtener el ID del receptor a partir del nombre ingresado
    string nombreReceptor = nombreOrderInput.text;
    if (string.IsNullOrEmpty(nombreReceptor))
    {
        Debug.LogError("El nombre del receptor no puede estar vacío.");
        yield break;
    }

    UnityWebRequest requestReceptor =
    UnityWebRequest.Get($"{baseUrl}/jugadores/obtener_id?nombre={nombreReceptor}");
    requestReceptor.SetRequestHeader("Authorization", "Bearer " + authToken); //
    Asegurate de enviar el token para seguridad

    yield return requestReceptor.SendWebRequest();

    if (requestReceptor.result == UnityWebRequest.Result.Success)
    {
        var jsonReceptor = JSON.Parse(requestReceptor.downloadHandler.text);
        int receptorId = jsonReceptor["id"].AsInt;

        // Proceder a enviar la solicitud de amistad
        StartCoroutine(EnviarSolicitud(receptorId, authToken));
    }
    else
    {
        Debug.LogError($"Error al obtener ID del receptor: {requestReceptor.error}");
        yield break;
    }
}

// Método para enviar la solicitud de amistad
private IEnumerator EnviarSolicitud(int receptorId, string authToken)
{
    string url = $"{baseUrl}/amistades/Enviar_solicitud";
    var datos = new JSONObject
    {
        ["nombre_receptor"] = nombreOrderInput.text // Usar el nombre del receptor
        como se espera en el backend
    };

    UnityWebRequest request = new UnityWebRequest(url, "POST");
    byte[] bodyRaw = System.Text.Encoding.UTF8.GetBytes(datos.ToString());
    request.uploadHandler = new UploadHandlerRaw(bodyRaw);
    request.downloadHandler = new DownloadHandlerBuffer();
    request.SetRequestHeader("Content-Type", "application/json");
}

```

```

request.SetRequestHeader("Authorization", "Bearer " + authToken);

yield return request.SendWebRequest();

if (request.result == UnityWebRequest.Result.Success)
{
    Debug.Log("Solicitud de amistad enviada correctamente.");
}
else
{
    Debug.LogError($"Error al enviar la solicitud de amistad: {request.error}");
}
}

// Método público para obtener las solicitudes pendientes
public void ObtenerPendientesPublic()
{
    StartCoroutine(ObtenerPendientes());
}
// Método para obtener las solicitudes pendientes
private IEnumerator ObtenerPendientes()
{
    string authToken = AuthController.GetAuthToken();
    if (string.IsNullOrEmpty(authToken))
    {
        Debug.LogError("No token encontrado. El usuario no está autenticado.");
        yield break;
    }

    string url = $"{baseUrl}/amistades/obtener_pendientes";
    UnityWebRequest request = UnityWebRequest.Get(url);
    request.SetRequestHeader("Authorization", "Bearer " + authToken); // Añadir token
a la cabecera

    yield return request.SendWebRequest();

    if (request.result == UnityWebRequest.Result.Success)
    {
        string jsonResponse = request.downloadHandler.text;
        var json = JSON.Parse(jsonResponse);
        var pendientesArray = json["pendientes"].AsArray;

        // Llamar al método para limpiar el panel de amigos antes de añadir nuevos
elementos
        PanelFriendsUI.ClearPanelFriendsUI();

        // Crear una lista de solo nombres y actualizar el UI
        foreach (var nodo in pendientesArray)
        {
            var item = nodo.Value; // Asegúrate de obtener el valor del nodo
            string nombre = item["nombre"];
            Debug.Log($"{nombre}");

            int solicitanteId = item["solicitante_id"].AsInt; // Obtener ID del
solicitante
            int receptorId = item["receptor_id"].AsInt; // Obtener ID del receptor
            Debug.Log($"{solicitanteId}");
            Debug.Log($"{receptorId}");
        }
    }
}

```

```

        // Crear un JSONObject para el jugador y actualizar el UI con el nombre
        JSONObject jugadorJson = new JSONObject(); // Usar JSONObject para crear
un nodo válido
        jugadorJson["nombre"] = nombre; // Asignar el nombre al JSONObject

        // Crear la entrada de la solicitud en el panel
        GameObject scoreEntry = PanelFriendsUI.UpdatePanelFriendsUI(jugadorJson);

        // Instanciamos los botones de aceptar y rechazar
        GameObject aceptarButton = Instantiate(AceptarButtonPrefab);
        GameObject rechazarButton = Instantiate(RechazarButtonPrefab);

        // Asignar las funciones a los botones utilizando los métodos existentes
        aceptarButton.GetComponent<UnityEngine.UI.Button>().onClick.AddListener(()
=> AceptarSolicitudPublic(solicitanteId, receptorId));

        rechazarButton.GetComponent<UnityEngine.UI.Button>().onClick.AddListener(() =>
RechazarSolicitudPublic(solicitanteId, receptorId));

        // Encontrar el contenedor específico de la entrada para los botones
        Transform entryTransform = scoreEntry.transform; // Esto es el transform
de la entrada

        // Añadir los botones al contenedor de la entrada
        aceptarButton.transform.SetParent(entryTransform);
        rechazarButton.transform.SetParent(entryTransform);

        // Ajustar las posiciones de los botones
        RectTransform aceptarRect = aceptarButton.GetComponent<RectTransform>();
        RectTransform rechazarRect = rechazarButton.GetComponent<RectTransform>();

        // Cambiar escala a 0.75 (25% más pequeños)
        aceptarRect.localScale = new Vector3(0.75f, 0.75f, 1);
        rechazarRect.localScale = new Vector3(0.75f, 0.75f, 1);

        // Colocar los botones a la derecha del nombre (si no usas un
HorizontalLayoutGroup)
        aceptarRect.anchoredPosition = new Vector2(0, -20); // Ajusta la
distancia a la derecha
        rechazarRect.anchoredPosition = new Vector2(100, -20); // Ajusta la
distancia a la derecha
    }
}
else
{
    Debug.LogError($"Error al obtener solicitudes pendientes: {request.error}");
}
}

public async void ObtenerAmigosAsync()
{
    if (PanelFriendsUI == null)
    {
        PanelFriendsUI = FindObjectOfType<PanelFriendsUI>();
        if (PanelFriendsUI == null)
        {
            Debug.LogError("No se encontró un PanelFriendsUI en la escena.");
        }
    }

    string authToken = AuthController.GetAuthToken();

```

```

if (string.IsNullOrEmpty(authToken))
{
    Debug.LogError("No token encontrado. El usuario no está autenticado.");
    return;
}

string url = $"{baseUrl}/amistades/obtener_amigos";
UnityWebRequest request = UnityWebRequest.Get(url);
request.SetRequestHeader("Authorization", "Bearer " + authToken); // Añadir token
a la cabecera

// Convertir el UnityWebRequest a una tarea asíncrona utilizando
TaskCompletionSource
await SendWebRequestAsync(request);

if (request.result == UnityWebRequest.Result.Success)
{
    string jsonResponse = request.downloadHandler.text;
    var json = JSON.Parse(jsonResponse);
    var amigosArray = json["amigos"].AsArray;

    // Limpiar el panel de amigos antes de añadir nuevos elementos
    PanelFriendsUI.ClearPanelFriendsUI();

    // Crear una lista de nombres y actualizar el UI
    foreach (var nodo in amigosArray)
    {
        var item = nodo.Value; // Asegúrate de obtener el valor del nodo
        string nombre = item["nombre"];

        // Crear un JSONObject para el jugador y actualizar el UI con el nombre
        JSONObject jugadorJson = new JSONObject(); // Usar JSONObject para crear
un nodo válido
        jugadorJson["nombre"] = nombre; // Asignar el nombre al JSONObject

        // Actualizar el panel de amigos con el nombre del jugador
        PanelFriendsUI.UpdatePanelFriendsUI(jugadorJson);
    }
}
else
{
    Debug.LogError($"Error al obtener la lista de amigos: {request.error}");
}
}

// Método para convertir UnityWebRequest en una tarea asíncrona
private Task SendWebRequestAsync(UnityWebRequest request)
{
    var tcs = new TaskCompletionSource<bool>();

    request.SendWebRequest().completed += (op) =>
    {
        if (request.result == UnityWebRequest.Result.Success)
        {
            tcs.SetResult(true); // Completa la tarea con éxito
        }
        else
        {
            tcs.SetException(new System.Exception(request.error)); // Completa con un
error
        }
    };
};

```

```

        return tcs.Task;
    }

// Método público para aceptar una solicitud de amistad
public void AceptarSolicitudPublic(int solicitanteId, int receptorId)
{
    StartCoroutine(AceptarSolicitud(solicitanteId, receptorId));
}

// Método para aceptar una solicitud de amistad
public IEnumerator AceptarSolicitud(int solicitanteId, int receptorId)
{
    string url = $"{baseUrl}/amistades/aceptar_solicitud";
    var datos = new JSONObject
    {
        ["solicitante_id"] = solicitanteId,
        ["receptor_id"] = receptorId
    };
    Debug.Log(datos);

    UnityWebRequest request = new UnityWebRequest(url, "POST");
    byte[] bodyRaw = System.Text.Encoding.UTF8.GetBytes(datos.ToString());
    request.uploadHandler = new UploadHandlerRaw(bodyRaw);
    request.downloadHandler = new DownloadHandlerBuffer();
    request.SetRequestHeader("Content-Type", "application/json");

    yield return request.SendWebRequest();

    if (request.result == UnityWebRequest.Result.Success)
    {
        Debug.Log("Solicitud aceptada correctamente");
    }
    else
    {
        Debug.LogError($"Error al aceptar solicitud: {request.error}");
    }
    this.ObtenerPendientesPublic();
}

// Método público para rechazar una solicitud de amistad
public void RechazarSolicitudPublic(int solicitanteId, int receptorId)
{
    StartCoroutine(RechazarSolicitud(solicitanteId, receptorId));
}

// Método para rechazar una solicitud de amistad
public IEnumerator RechazarSolicitud(int solicitanteId, int receptorId)
{
    string url = $"{baseUrl}/amistades/rechazar_solicitud";
    var datos = new JSONObject
    {
        ["solicitante_id"] = solicitanteId,
        ["receptor_id"] = receptorId
    };
};

```



```

UnityWebRequest request = new UnityWebRequest(url, "POST");
byte[] bodyRaw = System.Text.Encoding.UTF8.GetBytes(datos.ToString());
request.uploadHandler = new UploadHandlerRaw(bodyRaw);
request.downloadHandler = new DownloadHandlerBuffer();
request.SetRequestHeader("Content-Type", "application/json");

yield return request.SendWebRequest();

if (request.result == UnityWebRequest.Result.Success)
{
    Debug.Log("Solicitud rechazada correctamente");
}
else
{
    Debug.LogError($"Error al rechazar solicitud: {request.error}");
}
this.ObtenerPendientesPublic();
}

// Clase para representar una puntuación
public class Puntuacion
{
    public int jugador_id;
    public int puntos;
}

```

2. Interfaz de Usuario:

- **Diseño Responsivo:** Adaptación de la interfaz a diferentes resoluciones pero aun falta hacerlo responsivo para el caso de adaptar el juego a movil he implementar las mecanicas que solo seria encapsular los controles del jugador en varios metodos separando cada funcionalidad, comprobar el dispositivo y si es windows ejecutar el bloque de código que maneja los controles del jugador como hacemos actualmente y si es movil mostrar una interfaz de usuario en donde los botones llamen a los metodos para cada funcionalidad
- **Animaciones y Transiciones:** Mejora de la experiencia del usuario mediante efectos visuales atractivos. *Faltan por implementar*, lo mas parecido que tengo a este apartado es el sistema de notificaciones.

C.2. Fragmentos de Código del Cliente en Unity

7. Casos de Uso de los Usuarios

A continuación, se describen los principales casos de uso que involucran a los usuarios interactuando con el sistema.

7.1. Caso de Uso 1: Iniciar Sesión en la ventana de Login

Actor Principal: Jugador

Descripción: El usuario desea iniciar sesión en el sistema utilizando sus credenciales.

Flujo Básico:

1. El usuario accede a la pantalla de inicio de sesión.
2. El usuario ingresa su nombre de usuario y contraseña.
3. El usuario presiona "Iniciar sesión".
4. El sistema valida las credenciales ingresadas.
5. Si las credenciales son correctas, el sistema genera los tokens correspondientes y redirige al usuario a la pantalla principal del juego.

Flujo Alternativo:

- Error de Credenciales: Si las credenciales son incorrectas, el sistema muestra un mensaje de error y permite al usuario reintentar.

7.2. Caso de Uso 2: Recuperar Contraseña en la ventana de Login

Actor Principal: Usuario

Descripción: El usuario desea recuperar su contraseña olvidada introduciendo su correo electrónico.

Flujo Básico:

1. El usuario accede a la pantalla de inicio de sesión.
2. El usuario hace clic en "Recuperar contraseña".
3. El usuario ingresa su dirección de correo electrónico registrada.
4. El sistema valida que el correo electrónico esté registrado.
5. Si el correo electrónico es válido, el sistema envía un enlace para restablecer la contraseña al usuario.

Flujo Alternativo:

- Error de Correo Electrónico: Si el correo electrónico no está registrado, el sistema muestra un mensaje de error y permite al usuario reintentar.

7.3. Caso de Uso 3: Cambiar a la ventana de Registro desde la ventana de Login

Actor Principal: Usuario

Descripción: El usuario desea registrarse en el sistema y, por tanto, cambia a la pantalla de registro.

Flujo Básico:

1. El usuario accede a la pantalla de inicio de sesión.
2. El usuario hace clic en "Registrarse".
3. El sistema redirige al usuario a la pantalla de registro para que ingrese sus datos y cree una nueva cuenta.

7.4. Caso de Uso 4: Registrar una Cuenta en la ventana de Registro

Actor Principal: Jugador

Descripción: El usuario desea registrarse en el sistema desde la ventana de registro.

Flujo Básico:

1. El usuario accede a la pantalla de registro.
2. El usuario ingresa sus datos personales en los campos correspondientes y selecciona su nivel de privacidad.
3. El usuario presiona el botón "register".
4. El sistema valida los datos ingresados.
5. Si la validación es exitosa, el sistema registra la cuenta del usuario.
6. El sistema redirige al usuario a la pantalla de inicio de sesión y muestra un mensaje de éxito.

Flujo Alternativo:

- Error de Validación: Si los datos ingresados son incorrectos o faltan campos, el sistema muestra un mensaje de error y permite al usuario corregir la información.

7.5. Caso de Uso 5: Cambiar a la ventana de Login desde la ventana de Registro

Actor Principal: Usuario

Descripción: El usuario ya tiene una cuenta y desea cambiar a la pantalla de inicio de sesión desde la ventana de registro.

Flujo Básico:

1. El usuario accede a la pantalla de registro.
2. El usuario, si ya tiene una cuenta, presiona el enlace "login".
3. El sistema redirige al usuario a la pantalla de inicio de sesión.

7.6. Caso de Uso 6: Registrar una nueva Puntuación al Terminar la Partida

Actor Principal: Jugador

Descripción: El jugador finaliza una partida y se registra su puntuación en el Leaderboard.

Precondición:

- El jugador ha iniciado sesión y está autenticado correctamente mediante un token JWT.

Flujo Básico:

1. El jugador completa una partida en el juego.
2. El juego calcula la puntuación obtenida.
3. El cliente de Unity envía una solicitud POST /puntuación a la API con la puntuación y el ID del jugador.
4. El servidor valida la información y almacena la puntuación en la base de datos.
5. El servidor responde con una confirmación de éxito.
6. El cliente actualiza la interfaz del Leaderboard para reflejar la nueva puntuación registrada en caso de que el usuario no tuviese ninguna puntuación asociada.

Flujo Alternativo:

- Puntuación ya existente para el Usuario: Si el usuario ya tiene una puntuación en la base de datos, el juego actualiza la puntuación del jugador sobrescribiendo la anterior solo si la nueva es superior.
- Error de Conexión: Si la solicitud falla, el juego muestra un mensaje de error al jugador y ofrece reintentar el envío.

7.7. Caso de Uso 7: Consultar el Leaderboard General desde la ventana Menu

Actor Principal: Jugador

Descripción: El jugador desea ver el Leaderboard general para comparar su puntuación con otros jugadores.

Flujo Básico:

1. El jugador accede a la sección de Leaderboard en el Menu pulsando el botón *Leaderboard*.
2. El cliente de Unity envía una solicitud GET /puntuaciones?order_by=valor&order=desc a la API.
3. La API recupera las puntuaciones de la base de datos y las ordena de mayor a menor.
4. El servidor responde con la lista de puntuaciones.
5. El cliente muestra las puntuaciones en la interfaz del Leaderboard.

Flujo Alternativo:

- Filtros Aplicados: El jugador puede aplicar filtros (order), modificando el endpoint correspondiente a la solicitud **GET /puntuaciones** con parámetros adicionales (**order=desc** o **order=asc**). Para ello pulsa en los botones **order desc** y **order asc** respectivamente.

El jugador también puede aplicar filtros utilizando el endpoint

GET /puntuaciones/{nombre} para ver las puntuaciones por nombre de usuario. Para ello **escribe le nombre del usuario** y pincha en *search by name*

7.8. Caso de Uso 8: Ver el Perfil del Jugador desde la ventana Leaderboard

Actor Principal: Jugador

Descripción: El jugador desea ver su perfil con información detallada, como su nombre, email, número de teléfono y nivel de privacidad.

Precondición:

- El jugador ha iniciado sesión y está autenticado correctamente mediante un token JWT.

Flujo Básico:

1. El jugador pincha en perfil a través de la interfaz de usuario dentro de la sección Leaderboard.
2. El cliente de Unity envía una solicitud GET /jugadores/perfil a la API.
3. La API recupera los detalles del jugador desde la base de datos utilizando el user_id extraído del token JWT.
4. El servidor responde con la información del jugador (*jugadores.nombre, jugadores.email, jugadores.numTelefono, privacidad.nivel_de_privacidad*).
5. El cliente muestra la información detallada en la interfaz del perfil.
6. Una vez que el cliente a visto sus datos puede retroceder pulsando en **back**

Flujo Alternativo:

- Usuario no encontrado: Si el jugador no existe en la base de datos, el sistema responde con un error 404 Not Found y un mensaje indicando que el jugador no fue encontrado.

Este flujo esta definido por seguridad (captura de excepciones) pero realmente no es posible a no ser de que fuerces la escena *menu* desde el editor de *unity* sin pasar antes por la escena de autenticacion *menuIntro*

7.9. Caso de Uso 9: Actualizar Perfil del Jugador desde la ventana Leaderboard

Actor Principal: Jugador

Descripción: El jugador desea actualizar su información personal, como su nombre, email, número de teléfono o nivel de privacidad.

Precondición:

- El jugador ha iniciado sesión y está autenticado correctamente mediante un token JWT.

Flujo Básico:

1. El jugador pincha en perfil a través de la interfaz de usuario dentro de la sección Leaderboard.
2. El jugador actualiza los campos (*jugadores.nombre, jugadores.email, jugadores.numTelefono, privacidad.nivel_de_privacidad*) que desea cambiar y después pulsa en **update**.
3. El cliente de Unity envía una solicitud PUT /jugadores/perfil con los nuevos datos a la API.

4. La API valida que los datos recibidos son completos (nombre, email, número de teléfono y nivel de privacidad).
5. El servidor actualiza la información del jugador en la tabla jugadores.
6. El servidor actualiza también el campo nivel_de_privacidad en la tabla privacidad
7. Si la actualización es exitosa, el servidor responde con un mensaje de éxito y código 200 OK.
8. El cliente muestra una notificación indicando que el perfil se ha actualizado correctamente.

Flujo Alternativo:

- Datos incompletos: Si los datos enviados están incompletos, el sistema responde con un error 400 Bad Request y un mensaje indicando que los datos son incompletos.
- Error al actualizar: Si ocurre un error al intentar actualizar el perfil, el sistema responde con un error 500 Internal Server Error.

7.10. Caso de Uso 10: Añadir Amigos

Descripción: Permite a un usuario enviar una solicitud de amistad a otro usuario.

Actores:

- Jugador solicitante
- Sistema de Leaderboard
- Jugador receptor

Precondiciones:

- Ambos usuarios deben estar registrados en el sistema.
- El usuario solicitante no debe estar bloqueado por el usuario receptor.

Flujo Principal:

1. El usuario solicitante busca al usuario receptor mediante su *nombre*.
2. El usuario pulsa el botón **send friend request** y envía una **solicitud de amistad**.
3. El sistema guarda la solicitud de amistad en el estado "pendiente" y notifica al usuario receptor.

Postcondiciones:

- La solicitud de amistad queda registrada en el sistema.
- El usuario receptor puede ver la solicitud pendiente en su bandeja de solicitudes.

Flujo Alternativo:

- Si el usuario receptor ya está en la lista de amigos del usuario solicitante, el sistema muestra un mensaje indicando que la solicitud no es necesaria.

7.11. Caso de Uso 11: Aceptar Solicitudes de Amistad

Descripción: Permite a un usuario aceptar una solicitud de amistad pendiente de otro usuario.

Actores:

- Usuario receptor
- Sistema de Leaderboard

Precondiciones:

- El usuario receptor tiene una solicitud de amistad pendiente.

Flujo Principal:

1. El usuario receptor pulsa el botón **show friend request** para mostrar las solicitudes de amistad.
2. El sistema muestra las solicitudes pendientes.
3. El usuario selecciona una solicitud y elige la opción "Aceptar".
4. El sistema actualiza el estado de la solicitud de "*pendiente*" a "*aceptado*".
5. Ambos usuarios se añaden mutuamente a sus respectivas listas de amigos.

Postcondiciones:

- Los usuarios pasan a estar conectados como amigos.
- Los usuarios pueden ver su puntuación de juego siempre que tengan su *nivel_de_privacidad* en amigos o publico (en el caso de tenerlo en publico no haría falta una relación de amistad)

Flujo Alternativo:

- Si los usuarios intentan enviar otra solicitud de amistad a usuario con el que ya han establecido una relación de amistad previamente, el sistema muestra un mensaje indicando que la acción no es válida y la solicitud no es necesaria.

7.12. Caso de Uso 12: Rechazar Solicitudes de Amistad

Descripción: Permite a un usuario rechazar una solicitud de amistad pendiente de otro usuario.

Actores:

- Usuario receptor
- Sistema de Leaderboard

Precondiciones:

- El usuario receptor tiene al menos una solicitud de amistad pendiente.

Flujo Principal:

1. El usuario receptor abre la bandeja de solicitudes de amistad.
2. El sistema muestra las solicitudes pendientes.
3. El usuario selecciona una solicitud y elige la opción "Rechazar".
4. El sistema elimina la solicitud de amistad o la marca como "rechazada".
5. El sistema no permitirá una relación de amistad entre esos dos usuarios

Postcondiciones:

- La solicitud de amistad ya no aparece en la bandeja del usuario receptor.
- El usuario solicitante recibe una notificación del rechazo. (**falta por implementar**)

Flujo Alternativo:

- Si el usuario receptor intenta rechazar una solicitud ya aceptada o previamente rechazada, el sistema muestra un mensaje indicando que la acción no es válida.

7.13. Caso de Uso 13: Cerrar sesión desde pantalla Menu

Actor Principal: Jugador.

Descripción: El jugador desea cerrar sesión para salir del sistema de forma segura.

Precondición:

- El jugador ha iniciado sesión y está autenticado correctamente mediante un token JWT.

Flujo Básico:

1. El jugador desde el menu principal pulsa en el botón **back**.
2. El cliente de Unity invalida los *tokens* almacenados localmente.
3. El sistema redirige al usuario a la pantalla de inicio de sesión.

Flujo Alternativo:

4. Si hay problemas con la conexión al servidor, se elimina el token localmente y se informa al usuario.

7.14. Caso de Uso 14: Eliminar una Puntuación (Administración)

Actor Principal: Administrador del sistema (o servicio automatizado encargado del mantenimiento)

Descripción: El administrador del sistema desea eliminar una puntuación específica por motivos de integridad o incumplimiento de reglas.

Precondición:

- El administrador ha iniciado sesión y tiene el rol de admin para poder tomar su token y usarlo en la llamada a la api mediante *curl* para realizar la eliminación de datos.

Flujo Básico:

1. El administrador accede al panel de control administrativo.
2. Navega hasta la sección de gestión de puntuaciones.
3. Selecciona la puntuación a eliminar.
4. El cliente de administración envía una solicitud DELETE /puntuaciones/{id} a la API.
5. El servidor valida los permisos y elimina la puntuación de la base de datos.
6. El servidor responde con una confirmación de éxito.
7. El cliente actualiza la lista de puntuaciones para reflejar el cambio.

Flujo Alternativo:

- **Error de Eliminación:** Si ocurre un error al intentar eliminar la puntuación asociada al usuario (jugador), el sistema muestra un mensaje de error.
- **Permisos Insuficientes:** Si el usuario no tiene permisos de administrador, el servidor responde con un error de autorización y el cliente muestra un mensaje correspondiente.

7.15. Caso de Uso 15: Reinicio del Sistema de Puntuaciones al Final de una Temporada (Administración)

Actor Principal: Administrador del sistema (o servicio automatizado encargado del mantenimiento)

Descripción: En una aplicación que rastrea puntuaciones de usuarios o equipos, es necesario reiniciar las puntuaciones al final de cada temporada o torneo para comenzar de nuevo desde cero para la próxima temporada.

Precondición:

- El sistema de puntuaciones contiene datos de la temporada o evento anterior que ya no son necesarios para la siguiente temporada.
- El administrador ha iniciado sesión y tiene el rol de Admin para poder tomar su token y usarlo en la llamada al curl para realizar la eliminación de datos.

Escenario Principal:

1. El administrador elimina todos los jugadores mediante una instrucción curl a la API pasando su token personal que tiene el rol de admin.
2. El sistema envía una solicitud DELETE /puntuaciones/limpiar a la API.
3. El servidor recibe la solicitud y procede a eliminar todas las puntuaciones de la base de datos.
4. Si la eliminación es exitosa, el sistema responde con un código 200 OK.
5. El administrador recibe una notificación de éxito y el sistema está listo para la nueva temporada.

Flujo Alternativo:

- **Error de Eliminación:** Si ocurre un error al intentar eliminar las puntuaciones, el sistema muestra un mensaje de error.
- **Permisos Insuficientes:** Si el usuario no tiene permisos de administrador, el servidor responde con un error de autorización y el cliente muestra un mensaje correspondiente.

7.16. Caso de Uso 16: Reinicio del Sistema de Jugadores (Administración) (Para pruebas)

Actor Principal: Administrador del sistema (o servicio automatizado encargado del mantenimiento)

Descripción: En una aplicación que rastrea a los jugadores, es necesario reiniciar el registro de los jugadores al final del periodo de pruebas o para empezar de nuevo con un sistema limpio para la siguiente temporada.

Precondición:

- El sistema ha pasado el periodo de pruebas y está listo para funcionar o simplemente se quieren eliminar los usuarios al cambiar de temporada.

- El administrador ha iniciado sesión y tiene el rol de Admin para poder tomar su token y usarlo en la llamada al curl para realizar la eliminación de datos.

Escenario Principal:

1. El administrador elimina todos los jugadores mediante una instrucción curl a la API pasando su token personal que tiene el rol de Admin.
2. El sistema envía una solicitud DELETE /jugadores/limpiar a la API.
3. El servidor recibe la solicitud y procede a eliminar todos los jugadores de la base de datos.
4. Si la eliminación es exitosa, el sistema responde con un código 200 OK.
5. El administrador recibe una notificación de éxito y el sistema está listo para la nueva temporada.

Flujo Alternativo:

- **Error de Eliminación:** Si ocurre un error al intentar eliminar los jugadores, el sistema muestra un mensaje de error y el administrador puede intentar nuevamente.
- **Permisos Insuficientes:** Si el usuario no tiene permisos de administrador, el servidor responde con un error de autorización y el cliente muestra un mensaje correspondiente.

La base de datos **game**, implementada en **PostgreSQL**, contiene cuatro tablas principales: **puntuaciones**, **jugadores**, **privacidad** y **amistades**. A continuación, se presenta el diseño de ambas tablas y sus características.

8. BD game

8.1. Base de Datos game

La base de datos **game** está diseñada para gestionar la información de los jugadores y sus puntuaciones, así como su privacidad y las relaciones de amistad entre ellos. Cada tabla tiene una función específica en la gestión de los datos, lo que facilita la extensión y mejora del sistema en el futuro.

8.2. Tablas y Atributos

Tabla: Puntuaciones La tabla **puntuaciones** contiene los registros de puntuación de cada jugador.

Campo	Tipo de Datos	Descripción
id	INTEGER	Clave primaria, identifica cada registro de puntuación.
valor	INTEGER	Puntuación obtenida por el jugador.
jugador_id	INTEGER	Clave foránea, hace referencia a la tabla jugadores para identificar al jugador correspondiente.

Relación:

- La tabla **puntuaciones** tiene una relación **1:1** con la tabla **jugadores** mediante el campo **jugador_id**. Cada jugador tiene exactamente una puntuación.

Tabla: Jugadores La tabla **jugadores** almacena la información básica de los jugadores, como su nombre, correo electrónico, número de teléfono y otros datos de perfil.

Campo	Tipo de Datos	Descripción
id	INTEGER	Clave primaria, identifica cada jugador.
nombre	VARCHAR(12)	Nombre del jugador.
email	VARCHAR(50)	Dirección de correo electrónico del jugador.
numtelefono	VARCHAR(9)	Número de teléfono del jugador (opcional).
password	VARCHAR(256)	Contraseña del jugador (encriptada).
rol	VARCHAR(20)	Rol del jugador (por defecto 'jugador').

Relaciones:

- La tabla **jugadores** se relaciona con la tabla **puntuaciones** mediante el campo **id** (clave primaria).
- La tabla **jugadores** también se relaciona con las tablas **privacidad** y **amistades** mediante su **id**.

Tabla: Privacidad La tabla **privacidad** almacena la configuración de privacidad de cada jugador.

Campo	Tipo de Datos	Descripción
id	INTEGER	Clave primaria. Identifica cada registro de privacidad.
nivel_de_privacidad	ENUM('publico', 'privado', 'amigos')	Nivel de privacidad del jugador. Default 'publico'.
jugador_id	INTEGER	Clave foránea, hace referencia a la tabla jugadores .

Relaciones:

- La tabla **privacidad** tiene una relación **1:1** con la tabla **jugadores** mediante el campo **jugador_id**. Cada jugador tiene un único nivel de privacidad.

Tabla: Amistades La tabla **amistades** registra las solicitudes de amistad entre jugadores.

Campo	Tipo de Datos	Descripción
id	INTEGER	Clave primaria, identifica cada solicitud de amistad.
jugador_solicitante_id	INTEGER	Clave foránea, hace referencia a la tabla jugadores para el jugador solicitante.
jugador_receptor_id	INTEGER	Clave foránea, hace referencia a la tabla jugadores para el jugador receptor.
estado	VARCHAR(20)	Estado de la solicitud (por defecto 'pendiente').

Relaciones:

- La tabla **amistades** tiene una relación **1:N** con la tabla **jugadores** en ambas direcciones:

- Un jugador puede ser el solicitante de varias solicitudes de amistad.
- Un jugador puede recibir varias solicitudes de amistad.

8.3. Relaciones y Dependencias

- **Jugadores** → **Puntuaciones**: Relación **1:1** (cada jugador tiene una única puntuación). Esto está garantizado por la restricción de clave foránea `jugador_id` en la tabla **puntuaciones**, que hace referencia a **jugadores.id** con la restricción **UNIQUE**, asegurando que cada jugador tenga solo una puntuación.
- **Jugadores** → **Privacidad**: Relación **1:1** (cada jugador tiene un único nivel de privacidad). Esto se gestiona con la clave foránea `jugador_id` en la tabla **privacidad**, haciendo referencia a **jugadores.id**.
- **Jugadores** → **Amistades**: Relación **1:N** en ambas direcciones:
 - Un jugador puede enviar múltiples solicitudes de amistad (como `jugador_solicitante_id`).
 - Un jugador puede recibir múltiples solicitudes de amistad (como `jugador_receptor_id`).

La tabla **amistades** gestiona estas relaciones con las claves foráneas `jugador_solicitante_id` y `jugador_receptor_id`, que hacen referencia a **jugadores.id** en ambas direcciones.

8.4. Mantenimiento de la Base de Datos

Para asegurar la integridad y consistencia de los datos, el sistema implementa verificaciones de integridad referencial y mecanismos de validación de datos. Por ejemplo, se realiza una verificación al registrar puntuaciones para garantizar que el nombre del jugador no exista ya en la tabla **jugadores**.

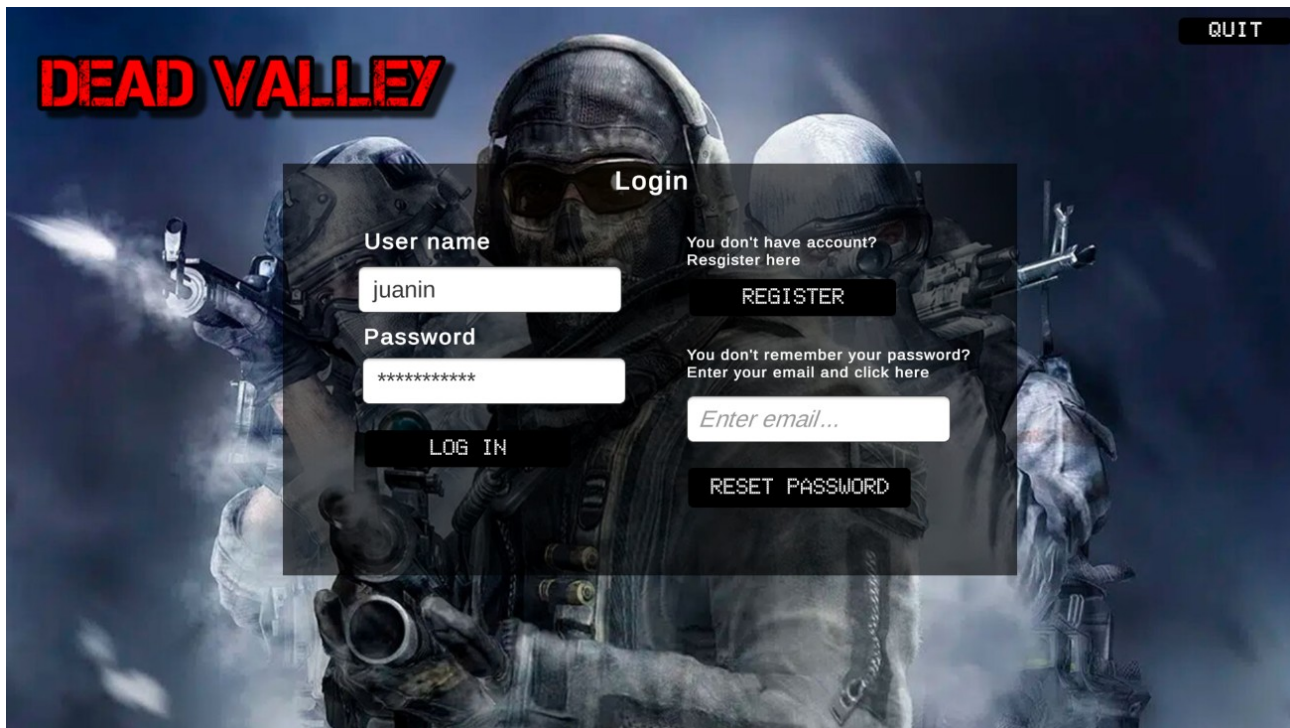
Este modelo de datos es flexible y escalable, lo que facilita la incorporación de nuevas funcionalidades, como la gestión de privacidad y de amigos.

B.2. Estructura de la Base de Datos PostgreSQL

9. Diseño de la interfaz

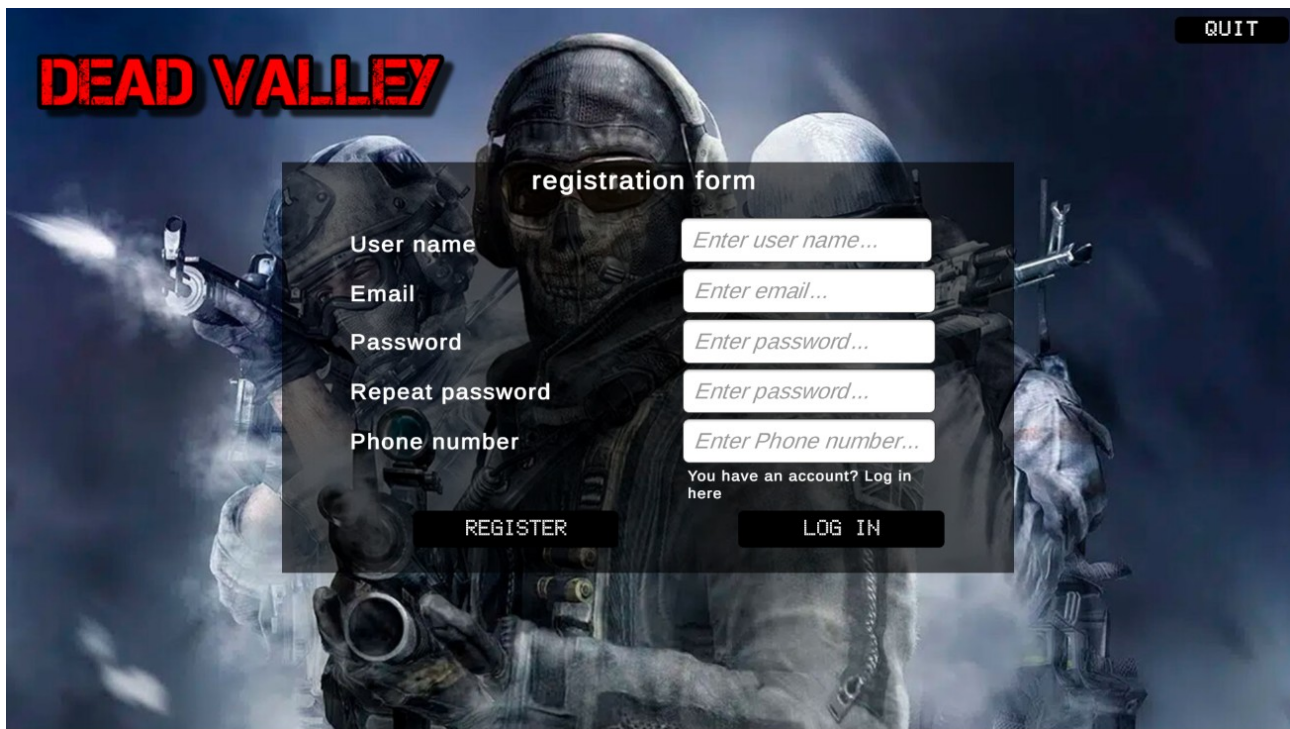
El diseño de la interfaz de usuario se enfoca en ofrecer una experiencia fluida e intuitiva para el jugador. La pantalla principal está compuesta por varios elementos clave, como:

1. Pantalla de logueo:



- **Botones de navegación:** Acceso a pantalla register mediante el botón register y a la pantalla inicio mediante el botón log in una vez tengamos los datos introducidos, también podremos recuperar nuestra contraseña introduciendo el email y pulsando el botón reset password

2. Pantalla de registro:



- **Botones de navegación:** Acceso a pantalla login y registro del usuario mediante el botón register una vez tengamos los datos introducidos

3. Pantalla de inicio:



.

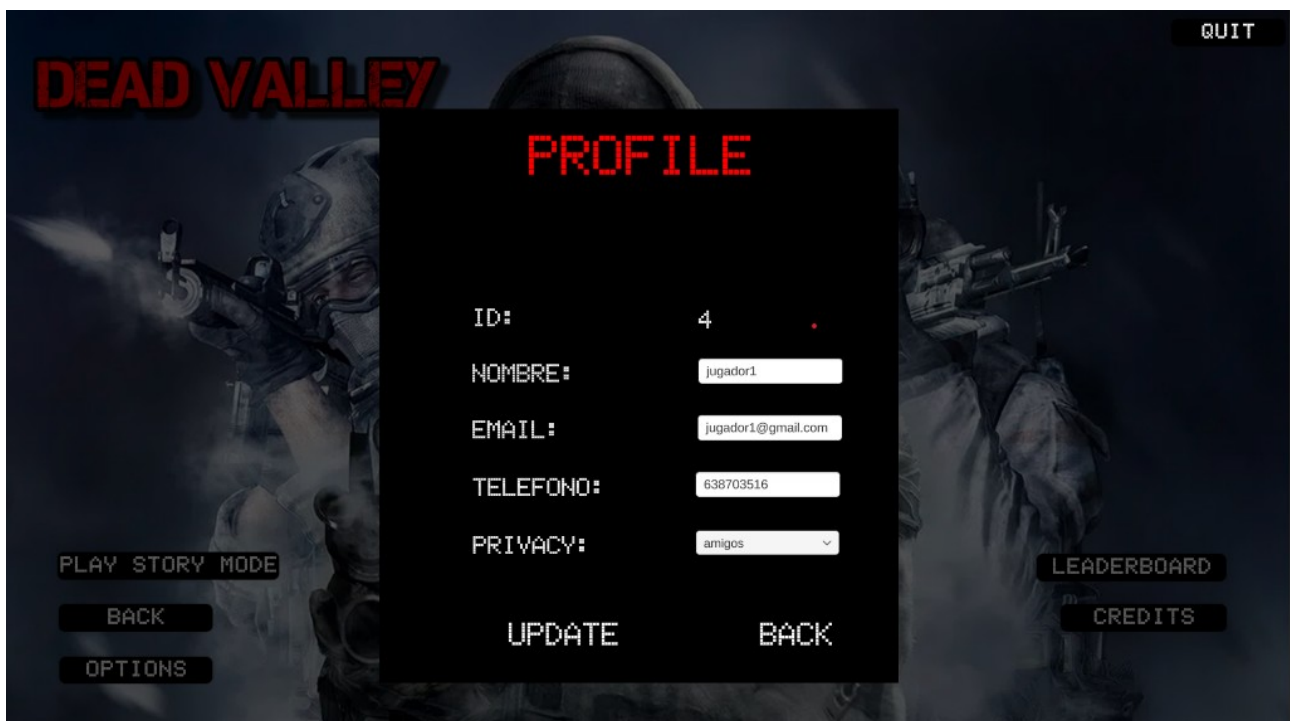
- **Botones de navegación:** Acceso a pantallas como jugar modo historia, opciones, tabla de puntuaciones, créditos y salida.

4. Pantalla de Leaderboard:



- **Botones de navegación:** Acceso a pantallas como Friends y perfil, además de la opción de volver al menú inicio, incluye también los botones para ordenar y buscar por nombre
- **Tabla de puntuaciones:** Muestra las puntuaciones más altas, ordenadas por el puntaje obtenido de mayor a menor y con el nombre del jugador.
- **Opciones de filtrado:** Permite ordenar la tabla por criterio mayor o menor puntuación.
- **Opciones de búsqueda:** Permite buscar la puntuación asociada a un jugador por su nombre.

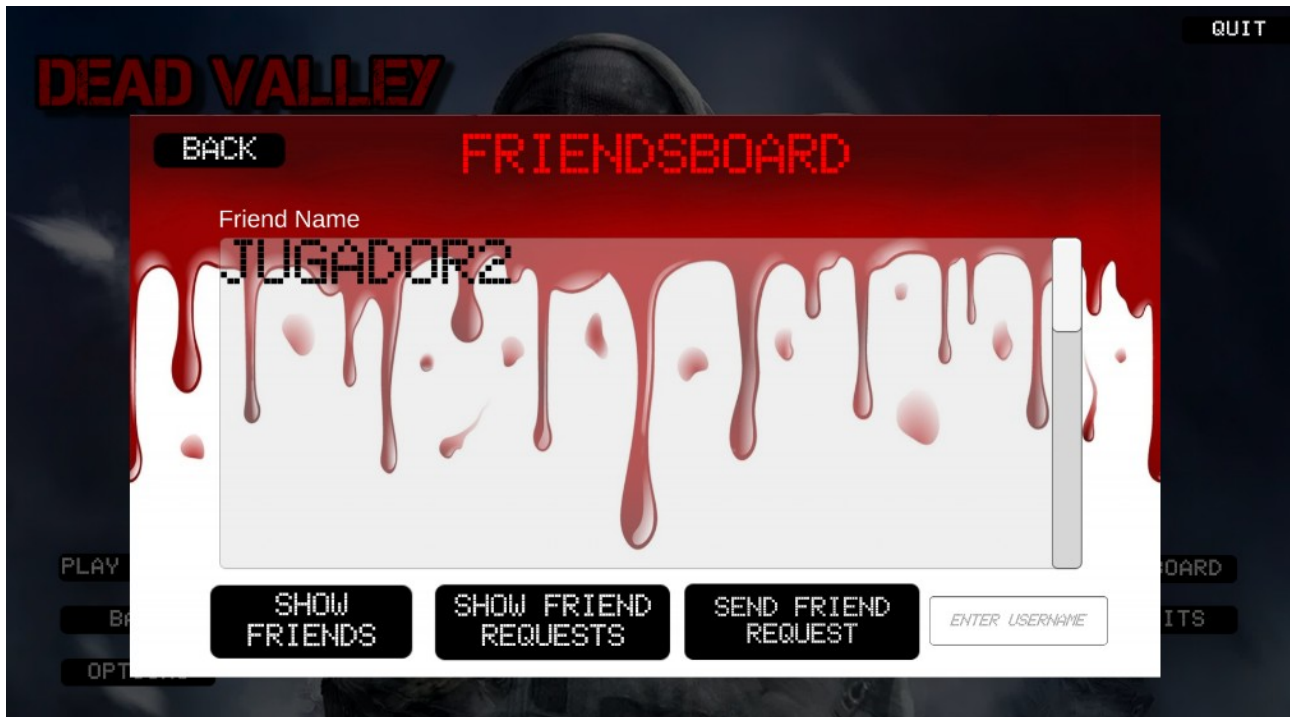
4.1 Pantalla de perfil de jugador:



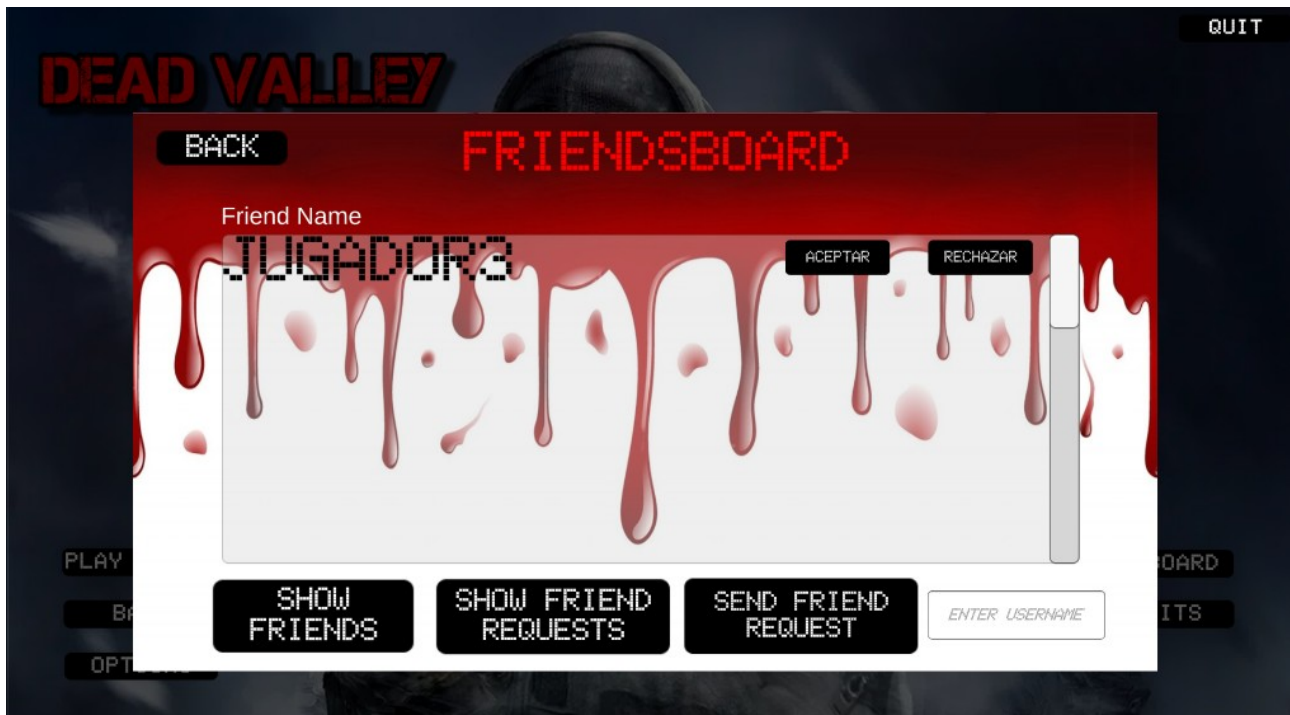
- **Botones de navegación:** Botón back para volver a la pantalla de Leaderboard y botón update para actualizar los datos del usuario en caso de que haya cambiado algún dato como el nombre el correo teléfono o el nivel de privacidad (publico, privado o amigos)
- **Detalles del jugador:** Muestra el nombre del jugador, sus datos email, numero de teléfono y nivel de privacidad, también permite modificar los mismos

4.2 Pantalla de Friendsboard

show friends:

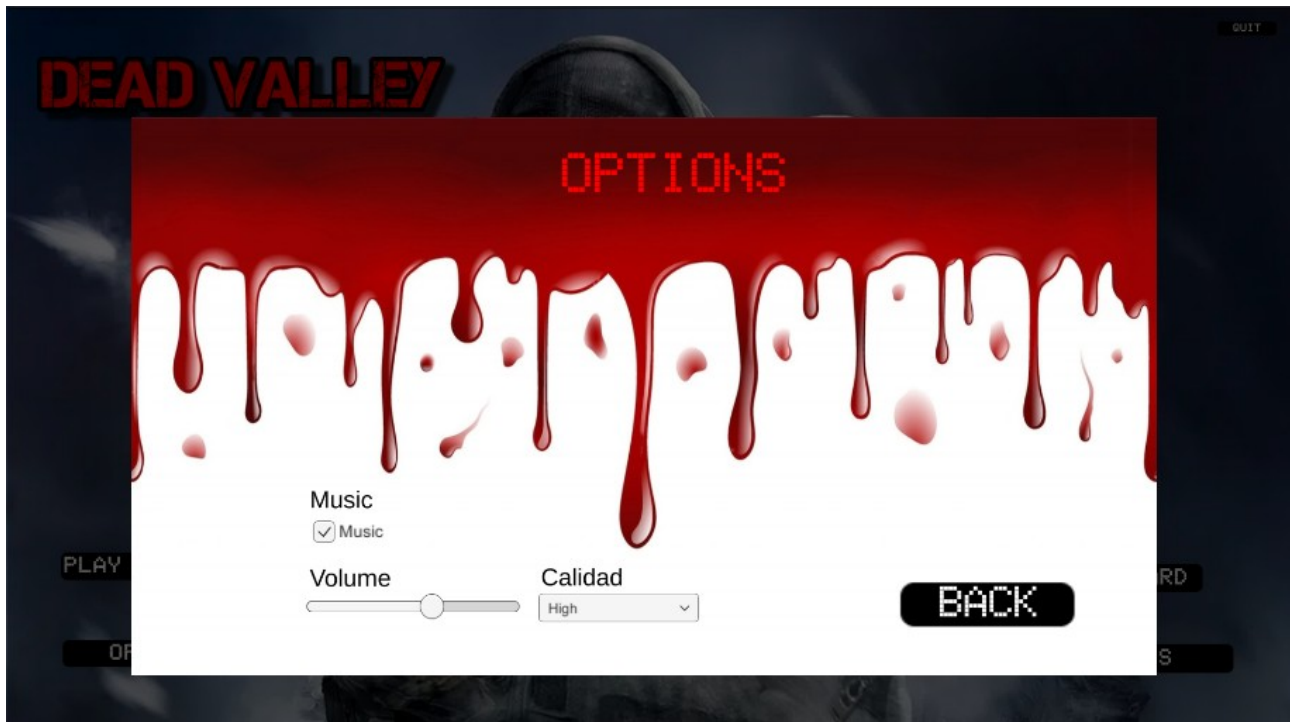


show friends request



- **Botones de navegación:** Botón back para volver a la pantalla de Leaderboard y botón show friends para ver tu lista de amigos, botón send friend request para enviar una petición de amistad basada en el nombre y botón show friend request para ver y contestar las solicitudes de amistad pendientes, para ello tenemos el botón aceptar o rechazar.
- **Tabla de amigos:** Muestra los amigos del jugador

5. Opciones



9.1 Implementación de las pantallas con funcionalidad parcial

La implementación de las pantallas de la interfaz está en una fase mas avanzada y cerca de estar terminado

La interacción entre las pantallas está operativa, pero algunas de las funcionalidades, como la comunicación en tiempo real con el servidor para obtener nuevas puntuaciones, están planificadas para la próxima fase de desarrollo.

9.2 Diagramas

Diagrama de clases final

El diagrama de clases refleja la estructura básica de las entidades en el sistema, donde se destacan las siguientes clases principales:

1. Jugador

- **Atributos:**
 - id: Integer (PK)
 - nombre: String (Unique, Not Null)
 - email: String (Unique)
 - numtelefono: String (Unique)
 - password: String

- `rol`: String (Default: 'jugador')
- **Relaciones:**
 - 1 **Jugador** → 1 **Puntuaciones**
 - 1 **Jugador** → 1 **Privacidad**
 - 1 **Jugador** → *N* **Amistades (Enviadas)** (Solicitante)
 - 1 **Jugador** → *N* **Amistades (Recibidas)** (Receptor)

Métodos:

- `__repr__()`: String

2. Puntuacion

- **Atributos:**
 - `id`: Integer (PK)
 - `valor`: Integer (Not Null)
 - `jugador_id`: Integer (FK, Not Null, Referencia a Jugador)
- **Relaciones:**
 - 1 **Puntuacion** → 1 **Jugador** (Relación con el jugador)

Métodos:

- `__repr__()`: String

3. Privacidad

- **Atributos:**
 - `id`: Integer (PK)
 - `nivel_de_privacidad`: Enum ('publico', 'privado', 'amigos') (Default: 'publico')
 - `jugador_id`: Integer (FK, Not Null, Referencia a Jugador)
- **Relaciones:**
 - 1 **Privacidad** → 1 **Jugador** (Relación con el jugador)

Métodos:

- `__repr__()`: String

4. Amistad

- **Atributos:**
 - `id`: Integer (PK)
 - `jugador_solicitante_id`: Integer (FK, Not Null, Referencia a Jugador)
 - `jugador_receptor_id`: Integer (FK, Not Null, Referencia a Jugador)
 - `estado`: String (Default: 'pendiente')
- **Relaciones:**
 - 1 **Amistad** → 1 **Jugador** (Solicitante)
 - 1 **Amistad** → 1 **Jugador** (Receptor)

Métodos:

- `__repr__()`: String

Diagrama de Arquitectura general A.2.

Diagrama Entidad-Relación

1. Entidad: Jugadores

- **Atributos:**
 - `id`: PRIMARY KEY
 - `nombre`: UNIQUE, NOT NULL
 - `email`: UNIQUE
 - `numtelefono`: UNIQUE
 - `password`
 - `rol`: DEFAULT 'jugador'

2. Entidad: Puntuaciones

- **Atributos:**
 - `id`: PRIMARY KEY
 - `valor`: NOT NULL
 - `jugador_id`: FOREIGN KEY, UNIQUE, NOT NULL (Referencia a `Jugadores.id`)

3. Entidad: Privacidad

- **Atributos:**
 - `id`: PRIMARY KEY
 - `nivel_de_privacidad`: ENUM ('publico', 'privado', 'amigos'), DEFAULT 'publico', NOT NULL
 - `jugador_id`: FOREIGN KEY, NOT NULL (Referencia a `Jugadores.id`)

4. Entidad: Amistades

- **Atributos:**
 - `id`: PRIMARY KEY
 - `jugador_solicitante_id`: FOREIGN KEY, NOT NULL (Referencia a `Jugadores.id`)
 - `jugador_receptor_id`: FOREIGN KEY, NOT NULL (Referencia a `Jugadores.id`)
 - `estado`: DEFAULT 'pendiente', NOT NULL

Relaciones

- **Jugadores** → **Puntuaciones**: Relación 1:1 (Cada jugador tiene una única puntuación, garantizado por `jugador_id` UNIQUE en `Puntuaciones`).
- **Jugadores** → **Privacidad**: Relación 1:1 (Cada jugador tiene un único nivel de privacidad).
- **Jugadores** → **Amistades**: Relación 1:N en dos direcciones (Un jugador puede enviar y recibir múltiples solicitudes de amistad).

```

+-----+
| Jugadores |
+-----+
| id (PK)    |
| nombre (UNIQUE, NOT NULL) |
| email (UNIQUE) |
| numtelefono (UNIQUE) |
| password   |
| rol (DEFAULT 'jugador') |
+-----+
+-----+
| Puntuaciones |
+-----+
| id (PK)      |
| valor (NOT NULL) |
| jugador_id (FK, UNIQUE, NOT NULL) -> Jugadores.id |
+-----+
+-----+
| Privacidad  |
+-----+
| id (PK)     |
| nivel_de_privacidad (ENUM: 'publico', 'privado', 'amigos', DEFAULT 'publico', NOT NULL) |
| jugador_id (FK, NOT NULL) -> Jugadores.id |
+-----+
+-----+
| Amistades   |
+-----+
| id (PK)     |
| jugador_solicitante_id (FK, NOT NULL) -> Jugadores.id |
| jugador_receptor_id (FK, NOT NULL) -> Jugadores.id |

```

| estado (DEFAULT 'pendiente', NOT NULL) |

+-----+

Relaciones:

1. Jugadores → Puntuaciones: Relación 1:1

- Un jugador tiene una única puntuación, garantizado por `jugador\_id UNIQUE` en Puntuaciones.

2. Jugadores → Privacidad: Relación 1:1

- Cada jugador tiene un único nivel de privacidad.

3. Jugadores → Amistades: Relación 1:N en dos direcciones

- Un jugador puede enviar y recibir múltiples solicitudes de amistad.

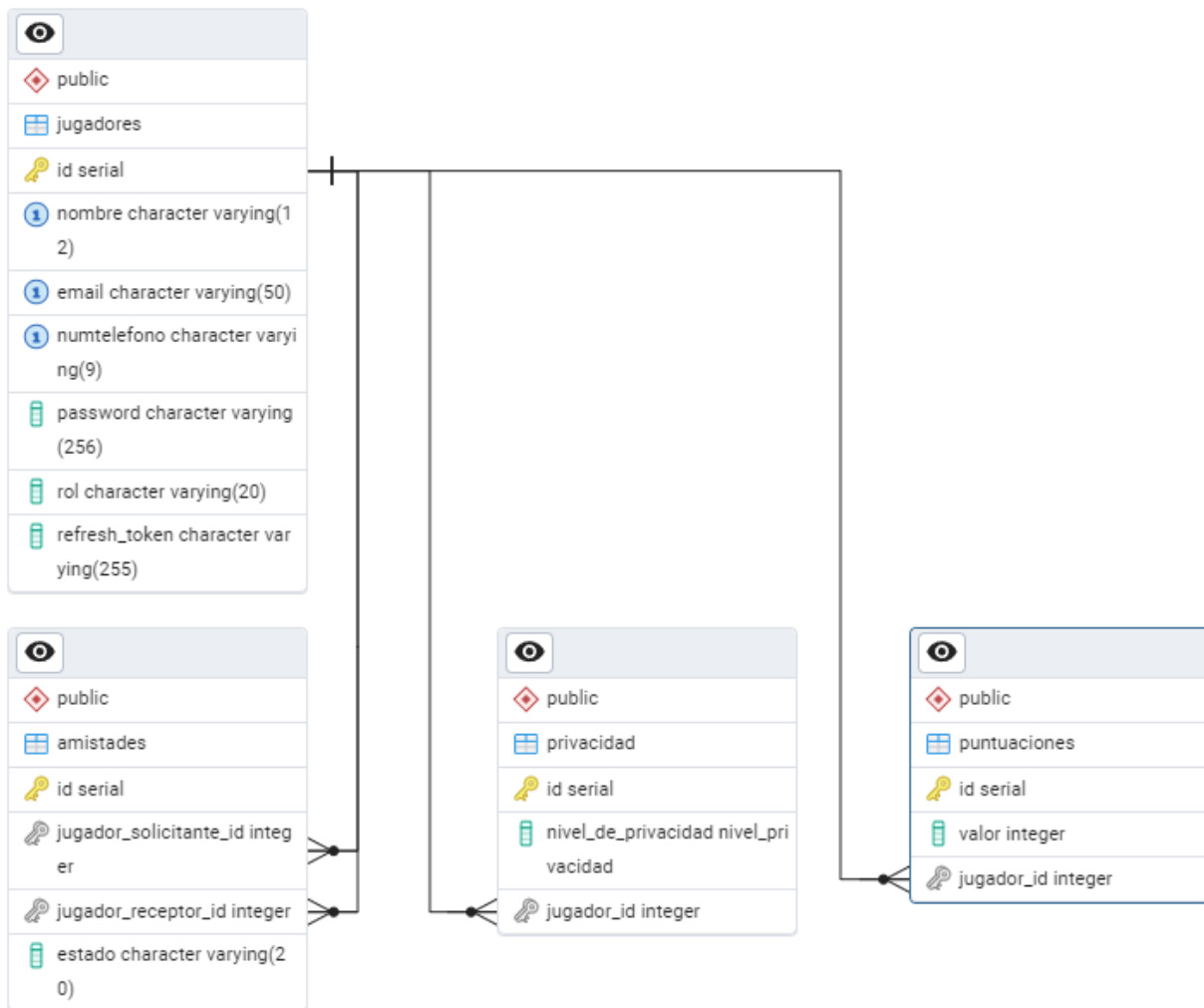


Diagrama entidad relacion A.3.

Modelo Relacional

Entidad: Jugadores

Tabla: jugadores

Esta tabla representa a los jugadores y sus atributos únicos.

Columna	Tipo de dato	Restricciones
id	SERIAL	PRIMARY KEY
nombre	VARCHAR(12)	UNIQUE, NOT NULL
email	VARCHAR(50)	UNIQUE
numtelefono	VARCHAR(9)	UNIQUE
password	VARCHAR(256)	
rol	VARCHAR(20)	DEFAULT 'jugador'

Entidad: Puntuaciones

Tabla: puntuaciones

Esta tabla almacena las puntuaciones de los jugadores, con una relación **1:1** entre jugadores y puntuaciones.

Columna	Tipo de dato	Restricciones
id	SERIAL	PRIMARY KEY
valor	INTEGER	NOT NULL
jugador_id	INTEGER	UNIQUE, NOT NULL, FOREIGN KEY (jugadores.id)

Entidad: Privacidad

Tabla: privacidad

Esta tabla define los niveles de privacidad de cada jugador, con una relación **1:1** entre jugadores y privacidad.

Columna	Tipo de dato	Restricciones
id	SERIAL	PRIMARY KEY
nivel_de_privacidad	ENUM ('publico', 'privado', 'amigos')	NOT NULL, DEFAULT 'publico'
jugador_id	INTEGER	NOT NULL, FOREIGN KEY (jugadores.id)

Entidad: Amistades

Tabla: amistades

Esta tabla almacena las relaciones de amistad entre jugadores, con una relación **1:N** en dos direcciones (un jugador puede enviar y recibir múltiples solicitudes).

Columna	Tipo de dato	Restricciones
id	SERIAL	PRIMARY KEY
jugador_solicitante_id	INTEGER	NOT NULL, FOREIGN KEY (jugadores.id)
jugador_receptor_id	INTEGER	NOT NULL, FOREIGN KEY (jugadores.id)
estado	VARCHAR(20)	NOT NULL, DEFAULT 'pendiente'

Relaciones en el modelo relacional

1. Relación Jugadores → Puntuaciones (1:1):

- Clave primaria de jugadores (id) referenciada como clave externa jugador_id en puntuaciones.
- Restricción UNIQUE en puntuaciones.jugador_id asegura la relación **1:1**.

2. Relación Jugadores → Privacidad (1:1):

- Clave primaria de jugadores (id) referenciada como clave externa jugador_id en privacidad.
- Restricción **1:1** mediante diseño del modelo.

3. Relación Jugadores → Amistades (1:N en dos direcciones):

- Claves externas jugador_solicitante_id y jugador_receptor_id en la tabla amistades referencian jugadores.id.
- Relación de **amistades** representa solicitudes de amistad y su estado.

Normalización del modelo relacional:

1. Primera Forma Normal (1NF):

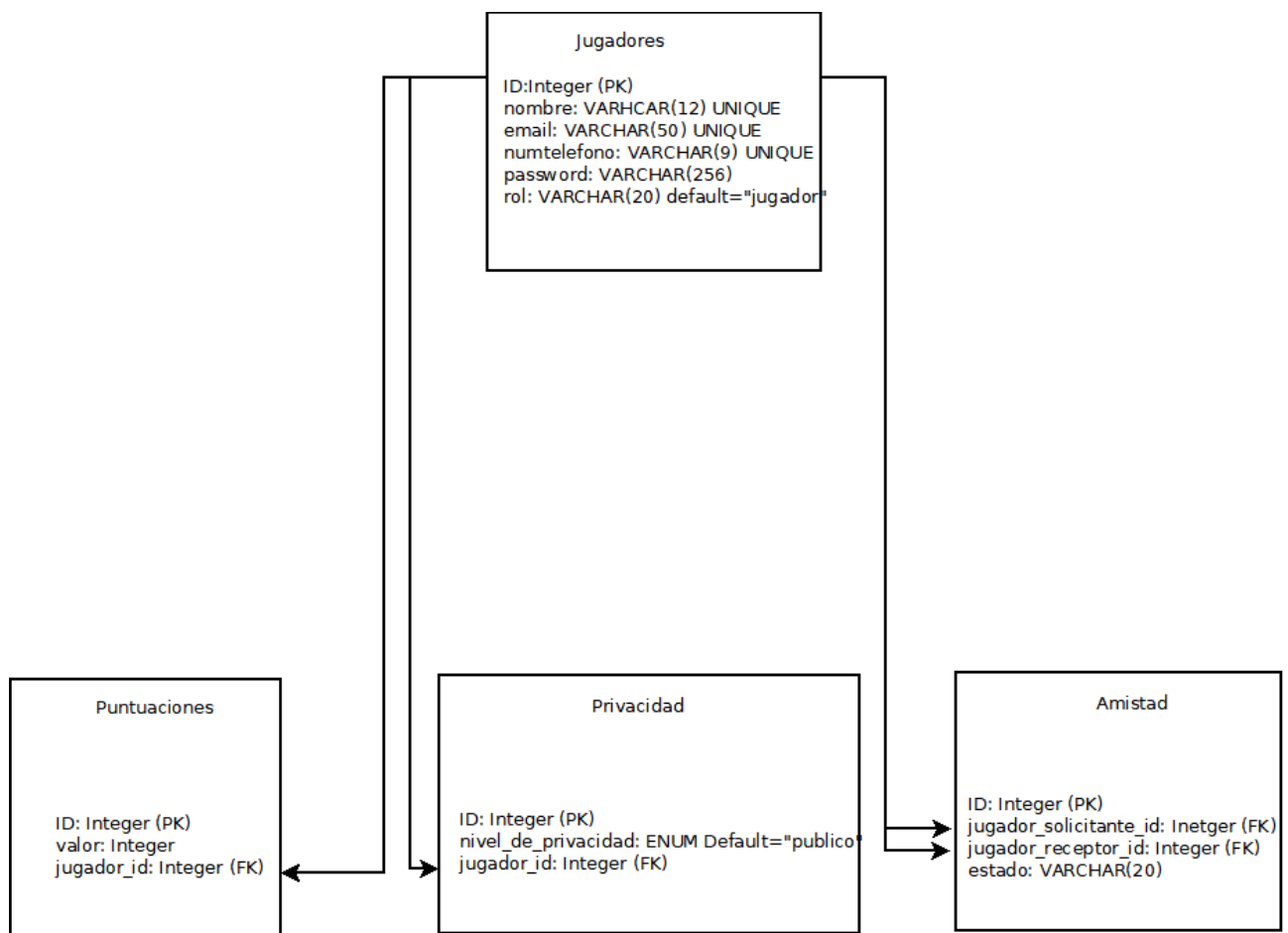
- Todas las tablas tienen valores atómicos y columnas únicas.

2. Segunda Forma Normal (2NF):

- Todas las tablas tienen claves primarias únicas y los atributos dependen únicamente de estas.

3. Tercera Forma Normal (3NF):

- No hay dependencias transitivas (todos los atributos no clave dependen directamente de la clave primaria de cada tabla).



Modelo Relacional A.4.

Modelo normalizado

1FN (Primera Forma Normal)

En este paso, se asegura que los datos están organizados en tablas y que cada atributo contiene valores atómicos.

Tablas:

Jugadores

- id (PK)
- nombre (UNIQUE, NOT NULL)
- email (UNIQUE)
- numtelefono (UNIQUE)
- password
- rol (DEFAULT 'jugador')

Puntuaciones

- id (PK)
- valor (NOT NULL)
- jugador_id (FK, UNIQUE, NOT NULL) → Jugadores(id)

Privacidad

- id (PK)
- nivel_de_privacidad (ENUM, DEFAULT 'publico', NOT NULL)
- jugador_id (FK, NOT NULL) → Jugadores(id)

Amistades

- id (PK)
- jugador_solicitante_id (FK, NOT NULL) → Jugadores(id)
- jugador_receptor_id (FK, NOT NULL) → Jugadores(id)
- estado (DEFAULT 'pendiente', NOT NULL)

2FN (Segunda Forma Normal)

En este paso, eliminamos la redundancia funcional asegurándonos de que todos los atributos no primarios dependen completamente de la clave primaria.

1. La tabla **Jugadores** ya cumple con 2FN, ya que todos sus atributos dependen únicamente de la clave primaria (**id**).
2. La tabla **Puntuaciones** también está en 2FN porque todos los atributos (en este caso, **valor**) dependen completamente de la clave primaria (**id**).
3. La tabla **Privacidad** está normalizada en 2FN ya que **nivel_de_privacidad** depende completamente de **id**.

4. En la tabla **Amistades**, todos los atributos dependen de **id**, por lo que también está en 2FN.

3FN (Tercera Forma Normal)

En este paso, eliminamos cualquier dependencia transitiva.

Análisis:

1. Ninguna tabla contiene dependencias transitivas, ya que:
 - En **Jugadores**, todos los atributos son independientes y solo dependen de la clave primaria.
 - En **Puntuaciones**, no hay atributos que dependan de otros atributos no clave.
 - En **Privacidad**, no hay dependencias transitivas.
 - En **Amistades**, todos los atributos son independientes.

Por lo tanto, el modelo ya cumple con la 3FN.

Modelo Normalizado Final

Tabla: Jugadores

Campo	Tipo	Restricciones
id	SERIAL	PRIMARY KEY
nombre	VARCHAR(12)	UNIQUE, NOT NULL
email	VARCHAR(50)	UNIQUE
numtelefono	VARCHAR(9)	UNIQUE
password	VARCHAR(256)	
rol	VARCHAR(20)	DEFAULT 'jugador'

Tabla: Puntuaciones

Campo	Tipo	Restricciones
id	SERIAL	PRIMARY KEY
valor	INTEGER	NOT NULL
jugador_id	INTEGER	UNIQUE, NOT NULL, FK → Jugadores(id)

Tabla: Privacidad

Campo	Tipo	Restricciones
id	SERIAL	PRIMARY KEY
nivel_de_privacidad	ENUM('publico', 'privado', 'amigos')	DEFAULT 'publico', NOT NULL

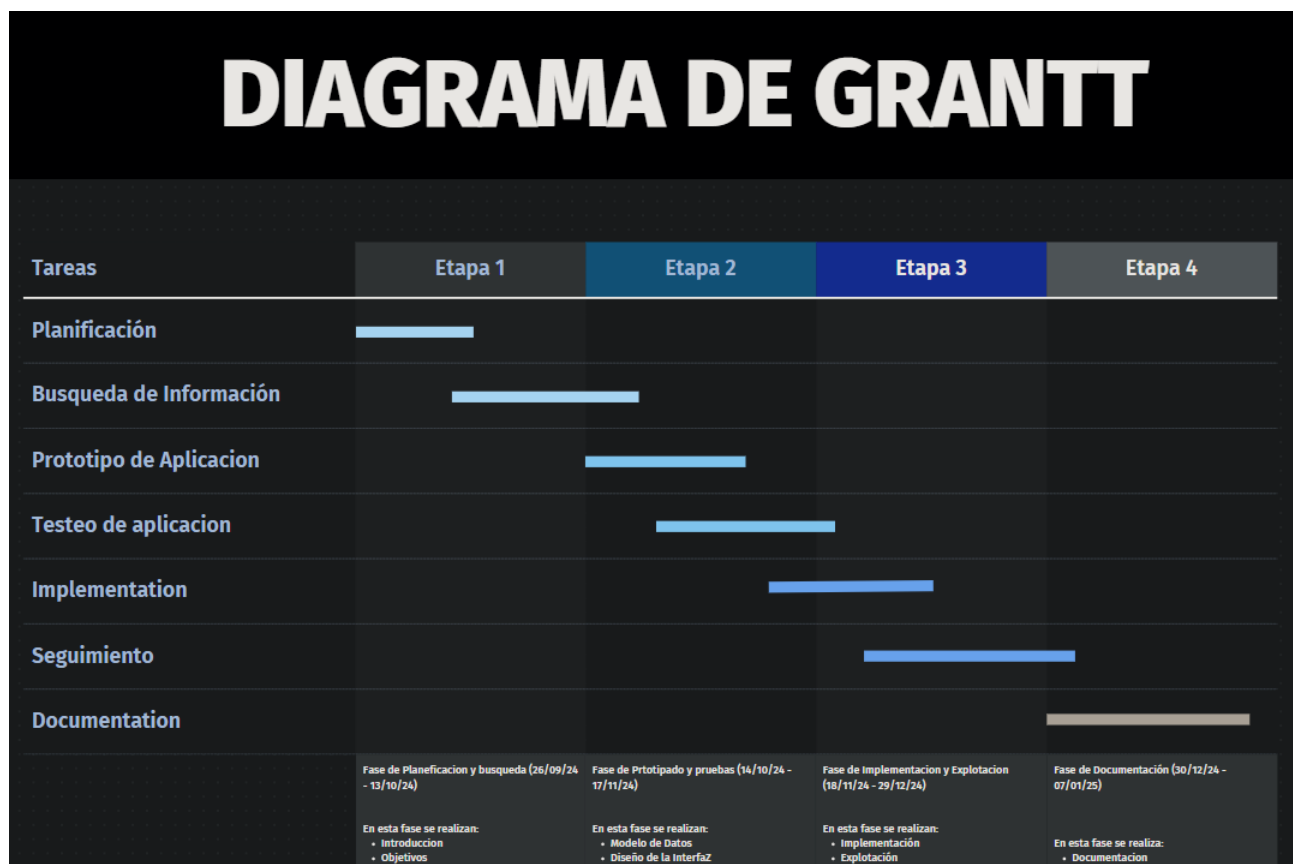
Campo	Tipo	Restricciones
jugador_id	INTEGER	NOT NULL, FK → Jugadores(id)

Tabla: Amistades

Campo	Tipo	Restricciones
id	SERIAL	PRIMARY KEY
jugador_solicitante_id	INTEGER	NOT NULL, FK → Jugadores(id)
jugador_receptor_id	INTEGER	NOT NULL, FK → Jugadores(id)
estado	VARCHAR(20)	DEFAULT 'pendiente', NOT NULL

Modelo Normalizado A.5.

Diagrama de Grantt



9.3 Planificación

La planificación del proyecto está dividida en varias fases con fechas tentativas de entrega para asegurar el progreso continuo:

1. Fase 1: Diseño y prototipado (Semana 1 - Semana 3)

- Diseño de la interfaz gráfica.
- Desarrollo de pantallas estáticas (pantalla de inicio, Leaderboard, perfil).
- Implementación de funcionalidad básica (entrada de nombre de jugador, visualización estática de puntuaciones).

2. Fase 2: Integración backend y frontend (Semana 4 - Semana 6)

- Integración de la API de Flask con el frontend de Unity.
- Implementación de funcionalidad para mostrar puntuaciones dinámicas desde el backend.
- Conexión con la base de datos PostgreSQL para almacenar y consultar puntuaciones.

3. Fase 3: Pruebas y optimización (Semana 7 - Semana 8)

- Pruebas unitarias y de integración de las funcionalidades.
- Optimización de la interfaz y el rendimiento.
- Implementación de medidas de seguridad como JWT y manejo de CORS.

4. Fase 4: Finalización y documentación (Semana 9 - Semana 10)

- Documentación del código y preparación del informe final.
- Últimas pruebas y entrega del sistema final.

10. Ejecución del Proyecto (Puesta en marcha)

1. Backend:

- Asegúrate de tener Flask y las dependencias necesarias instaladas.

```
pip install -r requirements.txt
```

- Configura una base de datos PostgreSQL y carga el esquema desde el script sql postgres.sql

o añadiendo :

la definición de los modelos de el archivo models.py en el archivo principal api.py

y las líneas que utilizan Flask-migrate

```
# Crear las tablas en la base de datos según los modelos definidos
with app.app_context():
    db.create_all()
```

si seguimos esta ultima opción habría que ejecutar manualmente las instrucciones del script postgres.sql referidas a la creación y concedida de permisos a los usuarios admin y client

- Ejecuta la aplicación con el comando:

```
python app.py
```

2. BD:

- Configuración de BD:
Lo primero es dirigirnos a la ruta: C:\Program Files\PostgreSQL\17\data
y buscar el archivo pg_hba.conf debemos editarlo o directamente sustituirlo por el que comparto en el proyecto dentro de la carpeta Implementar_leaderboard_en_otro_juego


```

#
# -----
# Put your actual configuration here
# -----
#
# If you want to allow non-local connections, you need to add more
# "host" records. In that case you will also need to make PostgreSQL
# listen on a non-local interface via the listen_addresses
# configuration parameter, or via the -i or -h command line switches.

# password de user admin = P@ssw0rd1?

# TYPE  DATABASE        USER            ADDRESS                 METHOD
# "local" is for Unix domain socket connections only
local   all             admin            password
local   game          client           password
local   all             all              reject

# IPv4 local connections:
host    all             admin            127.0.0.1/32            password
host    game          client           127.0.0.1/32            password
host    all             all              127.0.0.1/32            reject

# IPv6 local connections:
host    all             admin            ::1/128                 password
host    game          client           ::1/128                 password
# Allow replication connections from localhost, by a user with the
# replication privilege.
host    replication    all              ::1/128                 reject

```

lo siguiente ejecutar el script **PostgreSQL** el cual levantara la base de datos game sus tablas y relaciones, también creara un usuario **admin** con todos los privilegios sobre la bd game y sus tablas y otro **client** con permiso de escritura lectura, actualización y conexión en game y sus tablas, client se usara en la conexión con la base de datos desde la Api Flask y admin será usado en la realización de reportes desde java.

- El usuario admin que hemos creado en postgre tiene estas credenciales: admin = P@ssw0rd1?
- El otro usuario client que hemos creado en postgre tiene estas credenciales: client = sesamoPass1?

3. Cliente Unity:

- En caso de simplemente desplegar el juego actual ejecutamos el juego con la api arrancada y listo
- En caso de querer integrar el leaderboard en otro juego

Importa el archivo de Unity con el menú completo, los scripts y prefabs necesarios. realizar las asignaciones necesarias

seguimos los pasos que se definen en el archivo
Implementar_leaderboard_en_otro_juego.txt

finalmente

Configura el juego para que utilice el sistema de Leaderboard llamando a los métodos EnviarPuntuacion.

Ej:

```
LeaderBoardApi.instance.EnviaPuntuacion(player.playerScore * (int)player.time);
```

para usar nuestra **API** desde el juego Dead Valley o cualquier otro juego debemos configurar postgres17 y ejecutar postgres.sql para generar la BD Game, las tablas, usuarios, restricciones etc y después poner la **API** en funcionamiento

(y en caso de querer realizar la integración con otro juego) importaremos el paquete de unity y realizaremos las asignaciones que sean necesarias,

seguimos los pasos que se definen en el archivo Implementar_leaderboard_en_otro_juego.txt

Por ahora nuestro servidor esta montado en local ya que estamos haciendo pruebas con el servidor y el cliente en el mismo equipo y no hay problema por usar localhost en el cliente, pero en caso de querer probar a conectarnos a nuestra api desplegada en un servidor abierto a internet deberíamos cambiar *localhost* por la ip del servidor en el lado del cliente. Para ello la ip del servidor ha de estar configurada como estática y los puertos necesarios abiertos (5000 en este caso) .

```
private string apiUrl = "http://localhost:5000/puntuaciones";
```

finalmente (si estas realizar la integración con otro juego) habría que añadir esta linea desde el script donde se realice el conteo de puntos por partida:

```
LeaderBoardApi.instance.EnviaPuntuacion(puntuación_jugador);
```

1.guia uso de api – (para jugadores con rol de admin)

```
curl -X DELETE "http://localhost:5000/puntuaciones/<NOMBRE_DEL_JUGADOR>" \  
-H "Authorization: Bearer  
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6NywiZXhwIjoxNzE1NTk2MzQyYyUkKO8rxv4Ohds_yWXH7a9ah6ZGWgR5cqLy198"
```

```
curl -X DELETE "http://localhost:5000/puntuaciones/limpiar" \ -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6NywiZXhwIjoxNzM1NTk2MzQyfQ.97VyiUkKO8rxv4Ohds_yWXH7a9ah6ZGWgR5cqLy198"
```

```
curl -X DELETE "http://localhost:5000/jugadores/limpiar" \  
-H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6NywiZXhwIjoxNzM1NTk2MzQyfQ.97VyiUkKO8rxv4Ohds_yWXH7a9ah6ZGWgR5cqLy198"
```

en el lado de la api la gestión de el jugador que tiene rol de admin y puede realizar borrados se hace mediante un campo rol en la tabla jugadores por defecto este campo contiene el valor jugador, la única manera de generar un jugador con el campo rol admin es realizando un update directamente desde una instrucción sql a la base de datos:

garantizar rol de admin

UPDATE jugadores

SET rol = 'admin'

WHERE id = 20; //tu id

además para usar los endpoints de borrado debes realizar las llamadas a la api mediante curl o postman manualmente(como se muestra al comienzo de este punto 1) debido a que la interfaz no contiene métodos para realizar estas llamadas desde el cliente en unity, además, para garantizar la autenticación de el usuario con rol administrador se utiliza una función con decorador y @wraps(f) (esto mantiene la firma de la función original) llamada authenticate_token2(required_role="admin"):

de esta manera nos obliga a pasar desde la llamada al curl el access_token generado mediante el nombre y la contraseña durante el inicio de sesión autenticando al admin en caso de que este realmente tenga el rol admin

2. guía de uso de informes

para crear el informe ejecutar el programa java ubicado en la sección de informes

..\webapp_puntuaciones\informes\JasperReportExecutor desde NetBeans

o directamente ejecutando el .jar desde cmd

```
cd ..\webapp_puntuaciones\informes\JasperReportExecutor\target
```

```
java -jar JasperReportExecutor-1.0-SNAPSHOT.jar
```

Informes generados:

Informe de jugadores

jueves 02 enero 2025

Jugador ID	Nombre	Email	Telefono	Nivel de privacidad	Valor puntuacion
5	jugador2	jugador2@gmail.com	638703511	amigos	99
4	jugador1	jugador1@gmail.com	638703516	amigos	99

jueves 02 enero 2025

Page 1 of 1

Informe de puntuaciones

jueves 02 enero 2025

Puntuacion ID	Nombre	Valor puntuacion
2	jugador2	99
3	jugador1	99

jueves 02 enero 2025

Page 1 of 1

11. Explotación del Proyecto

1. Despliegue del Backend:

Para la puesta en producción del backend, se recomienda seguir estos pasos adicionales:

- **Configuración de entorno:** En un entorno de producción, es importante configurar las variables de entorno para proteger información sensible, como las credenciales de la base de datos. Utiliza un archivo `.env` o un servicio de gestión de configuraciones para almacenar valores como:

```
DATABASE_URL=postgresql://user:password@localhost/dbname
JWT_SECRET_KEY=your-secret-key
```

- **Despliegue en servidor:** Si deseas desplegar la aplicación Flask en un servidor de producción, una opción es usar un servidor WSGI como **Gunicorn** junto con **Nginx** como proxy inverso.

1. ¿Qué implica?

- **Gunicorn:** Es un servidor WSGI que se utiliza para ejecutar aplicaciones Flask (u otras aplicaciones Python compatibles con WSGI) en un entorno de producción. Permite manejar múltiples solicitudes simultáneamente (a través de "workers").
- **Nginx:** Es un servidor proxy inverso que actúa como intermediario entre los clientes y Gunicorn. Nginx es responsable de gestionar el tráfico HTTP/HTTPS, balancear la carga, servir archivos estáticos y manejar la seguridad.

2. Ventajas principales:

- Aumenta la **eficiencia y escalabilidad** al manejar múltiples solicitudes concurrentes.
- Permite una configuración **segura y profesional** para servicios que estarán disponibles al público.
- Nginx puede actuar como proxy inverso, gestionando certificados SSL (HTTPS) y mejorando la seguridad de la aplicación.

3. Contexto de uso:

- Recomendado para aplicaciones en **entornos de producción** donde esperas múltiples usuarios accediendo a tu API simultáneamente.
- Es ideal si el servidor tiene una dirección IP pública configurada y está preparado para estar en internet.

4. Ejemplo: Usarías Gunicorn con un comando como este para lanzar tu aplicación Flask:

```
gunicorn -w 4 app:app
```

Esto ejecuta 4 "workers" para manejar solicitudes concurrentes. Nginx redirige las solicitudes entrantes a Gunicorn.

podemos usar otras formas también como simplemente enfocarnos en **hacer que nuestra API sea accesible desde cualquier lugar**, partiendo de la **infraestructura de nuestra red local y expandiéndola hacia internet**. Para acceder a una API desde cualquier punto fuera de tu red local, necesitas una dirección IP externa (pública) que permita a los dispositivos de internet comunicarse con tu servidor. Vamos a clarificar cómo funciona esto:

Requisitos para que la API sea accesible desde cualquier lugar:

Dirección IP pública

Una dirección IP pública es necesaria para exponer la API al internet. Normalmente, el proveedor de servicios de internet (ISP) nos asigna una. Si estamos trabajando desde casa, probablemente nuestra red tenga una IP pública dinámica que cambia con el tiempo, pero también podemos contratar una IP pública estática si necesitas que sea fija.

Reenvío de puertos (port forwarding)

Si nuestro servidor está detrás de un router, debemos configurar el reenvío de puertos para redirigir las solicitudes entrantes desde la IP pública hacia nuestro servidor local. Esto se hace en el panel de administración del router:

Redirigimos las solicitudes que lleguen al puerto de la API (por ejemplo, 5000) hacia la dirección IP local de tu máquina (por ejemplo, 192.168.1.20).

Configuración del servidor

El servidor Flask debe estar configurado para escuchar en todas las interfaces de red (0.0.0.0) y en el puerto correspondiente. Ejemplo:

```
app.run(host='0.0.0.0', port=5000)
```

Firewall

Nos aseguramos de que el puerto que usa nuestra API esté abierto tanto en el firewall de tu sistema operativo como en el del router.

Limitaciones sin una IP pública:

Si no tienes una IP pública (ya sea estática o dinámica), la API solo será accesible desde dentro de tu red local. En este caso:

La dirección 192.168.x.x es una dirección privada y no puede ser alcanzada desde internet. Incluso configurando Flask para escuchar en 0.0.0.0, el alcance seguirá limitado a dispositivos dentro de tu red.

Opciones adicionales para exponer la API sin IP pública:

Otra forma de hacerlo sin usar una ip publica son los

Servicios de túneles (tunneling):

Herramientas como ngrok o localtunnel permiten exponer temporalmente tu API a través de

una dirección accesible desde cualquier lugar sin necesidad de configurar una IP pública. Esto es ideal para pruebas rápidas.

Alojamiento en un servidor remoto:

Sube tu aplicación a un servicio de hosting (como AWS, Heroku, o DigitalOcean) para que esté accesible desde cualquier lugar. Estos servicios te proporcionan una IP pública o un dominio para tu API.

En resumen, sin una IP pública o túnel, tu API será inaccesible fuera de la red local. Por tanto, si planeas ofrecer tu servicio a usuarios externos, es imprescindible configurar correctamente una IP pública estática con port forwarding, o considerar una solución de hosting o túneles.

- **Monitoreo y mantenimiento:** Se recomienda configurar sistemas de monitoreo, como **Prometheus** o **New Relic**, para asegurarse de que el sistema esté funcionando correctamente. También es recomendable configurar alertas por correo electrónico o en aplicaciones como **Slack** para recibir notificaciones sobre errores o caídas del servidor.

2. Explotación del Cliente Unity:

En la explotación del cliente Unity, los siguientes pasos son importantes para asegurar la correcta interacción entre el juego y el backend:

- **Conexión segura:** En un futuro podría asegurarse que las comunicaciones entre el cliente y el servidor estén cifradas utilizando **HTTPS**. Si estás utilizando un servidor dedicado, configura correctamente el certificado SSL para garantizar que los datos del usuario estén protegidos.
- **Actualización del sistema:** Realiza actualizaciones periódicas en el cliente Unity para mantener el juego actualizado con los cambios en el backend. Cuando se realicen modificaciones en el sistema de Leaderboard o nuevas características sean implementadas, asegúrate de actualizar el cliente y probar las interacciones con el servidor.
- **Escalabilidad y optimización:** Si el juego gana popularidad, será necesario hacer pruebas de carga en el backend para asegurarse de que pueda manejar un número creciente de usuarios. Optimiza el código del backend y el cliente para soportar un mayor tráfico sin afectar la experiencia de usuario.
- **Pruebas:** Para las pruebas existe un objeto incluido llamado **ScriptPruebas** que tiene añadido un script llamado **ProbarPost**, el cual puntúa una puntuación aleatoria entre 1 y 1000 con el usuario actual cada vez que pulsamos *space* desde el menu del juego (después de haber iniciado sesión correctamente).

Algunos pasos para añadir protección extra serian:

Para proteger tu API contra inyecciones SQL al usar Flask y SQLAlchemy, debes seguir una serie de buenas prácticas de seguridad. Aquí te detallo lo que debes implementar:

1. Usar Consultas Parametrizadas

SQLAlchemy ya previene inyecciones SQL al usar consultas parametrizadas por defecto. Por ejemplo, usa el siguiente formato:

```
# Seguro: consulta parametrizada
db.session.execute(
    text("SELECT * FROM usuarios WHERE id = :user_id"),
    {"user_id": user_id}
)
```

Evita concatenar cadenas manualmente para formar tus consultas SQL, como:

python

CopiarEditar

```
# Inseguro: susceptible a inyecciones SQL
db.session.execute(f"SELECT * FROM usuarios WHERE id = {user_id}")
```

2. Validar y Sanitizar Datos de Entrada

Aunque SQLAlchemy previene la mayoría de las inyecciones SQL, debes validar los datos de entrada para evitar datos inesperados. Usa librerías como `marshmallow` o `pydantic` para validar los esquemas de datos que llegan a tus endpoints.

```
from marshmallow import Schema, fields
```

```
class UserSchema(Schema):
    id = fields.Int(required=True)
```

```
# Validar datos de entrada
```

```
input_data = UserSchema().load(request.json)
```

3. Evitar Consultas Sincrónicas Directas

En lugar de escribir consultas SQL manuales, usa las funcionalidades del ORM de SQLAlchemy. Esto evita errores de formato o construcción insegura de consultas.

```
# ORM seguro
usuario = Usuario.query.filter_by(id=user_id).first()
```

4. Configurar Roles y Permisos

Implementa una estrategia de roles y permisos para limitar el acceso a recursos específicos. Por ejemplo:

- Asegúrate de que los usuarios puedan acceder solo a sus propios datos.
- Implementa control de acceso basado en el rol del usuario.

```
if current_user.role != "admin":
    return {"error": "Access denied"}, 403
```

5. Escapar Datos al Mostrar Resultados

Si estás mostrando datos en HTML o algún formato visible por el cliente, utiliza mecanismos para escapar las salidas y prevenir vulnerabilidades XSS.

6. Usar una Base de Datos Segura

Asegúrate de que tu base de datos esté configurada correctamente:

- Usa usuarios con permisos mínimos.
- No ejecutes tu aplicación como un usuario con privilegios de superusuario.
- Configura firewalls y acceso limitado por IP.

7. Registrar y Manejar Excepciones

Implementa manejo de excepciones para capturar y responder de forma adecuada a errores inesperados en las consultas.

```
try:
    usuario = Usuario.query.filter_by(id=user_id).first()
except Exception as e:
    app.logger.error(f"Error al consultar usuario: {e}")
    return {"error": "Error interno del servidor"}, 500
```

8. Habilitar Protección CSRF

Si tu API tiene formularios o tokens que interactúan con el cliente, habilita la protección CSRF (Cross-Site Request Forgery). Flask-WTF puede ayudarte con esto.

9. Usar Seguridad HTTPS

Configura tu servidor para usar SSL/TLS, asegurando que toda la comunicación con la API esté cifrada.

10. Limitar la Exposición de Información

No devuelvas información sensible como:

- Stack traces detallados en errores.
- Datos innecesarios en las respuestas de la API.

11. Hacer Pruebas de Seguridad

Usa herramientas como **sqlmap** para probar la robustez de tu API contra ataques de inyección SQL.

D.1. Guía para Desarrolladores

- La guía desde el lado del jugador es simple una vez entres en la aplicación vas a poder loguearte, registrarte o recuperar contraseña, una vez hayamos iniciado sesión entraremos al menú principal desde donde podremos jugar, modificar los ajustes, ver el Leaderboard y los créditos.

si jugamos una partida al terminarla se enviara un post a la api con nuestra puntuación que creara (en caso de que no exista puntuación asociada a ese user id) o actualizara (en caso de

que la nueva puntuación sea superior a la anterior establecida) un registro en la tabla puntuaciones asociado a tu id de usuario.

Si pinchamos en Leaderboard tendremos un botón **Back** para volver atrás y además podremos ver las puntuaciones del top mundial y ordenar en orden descendente (por defecto) o en orden ascendente pulsando en los botones **Order desc** y **Order asc** respectivamente, también tendremos un **cuadro de texto** donde podremos introducir el nombre de un jugador para filtrar y al pulsar en el botón **search by name** que solo nos muestre la puntuación del jugador introducido,

por otra parte en la parte superior derecha tenemos los botones **Friends** y **Profile**

desde el menú **Friendsboard** (botón **Friends**) veremos nuestra lista de **amigos**, disponemos de un botón **back** para volver atrás, luego disponemos de un botón **show friend request** que nos muestra las **solicitudes de amistad pendientes** y nos permite aceptarlas o rechazarlas mediante un par de botones **Aceptar** y **Rechazar** que se generan dinámicamente por código dentro de cada registro de la tabla, por otra parte si hemos aceptado a un amigo y queremos ver si ya se nos muestra en la lista de amigos tenemos el botón **show friends** que nos mostrara de nuevo la lista de amigos.

Finalmente tenemos un campo de texto donde podemos introducir el nombre de usuario de nuestro amigo y junto a él un botón de nombre send friend request que como su propio nombre indica sirve para enviar solicitudes de amistad a un amigo.

Desde el menú **Profile** (botón **Profile**) se nos muestran los datos de registro de nuestro usuario permitiéndonos volver atrás en cualquier momento mediante el botón **back** o **actualizar** los **datos de usuario** con **nuevos valores**.

D.2. Guía para Jugadores

12. Gestión económica o plan de empresa

12.1. Presupuesto estimado:

Costos de desarrollo: El desarrollo del proyecto no requiere de herramientas de pago, ya que se utilizarán tecnologías y herramientas de código abierto. El entorno de desarrollo será gratuito,

utilizando herramientas como Unity, PostgreSQL y Flask, todas ellas de acceso libre. No se prevé la compra de licencias ni software adicional en esta etapa.

Infraestructura: El proyecto inicialmente no contará con un servidor dedicado, por lo que se utilizará mi PC personal como servidor local. Se configurará una IP estática para que las llamadas del cliente se realicen hacia mi dirección IP, lo que facilitará las pruebas y el acceso remoto durante el desarrollo. Esto evitará la necesidad de pagar por servidores o servicios en la nube en esta fase. A medida que el proyecto avance y si se requiere mayor escalabilidad, se podrán considerar opciones como AWS o Google Cloud, dependiendo de las necesidades y recursos disponibles.

Publicidad y marketing: Dado que el proyecto es principalmente un trabajo académico o personal, no se contempla un presupuesto para publicidad o marketing en esta fase. Si en el futuro el juego tiene un alcance mayor y se decide lanzar al público, se podrían considerar opciones como campañas en redes sociales o plataformas de juegos, pero esto dependerá de la viabilidad del proyecto en ese momento.

Mantenimiento: El mantenimiento será gestionado de forma interna y será mínimo durante la fase de desarrollo, dado que el proyecto estará en constante evolución. A largo plazo, se podrían generar gastos asociados a la actualización de servidores o servicios si el proyecto pasa a un entorno de producción, pero por ahora no se estima un coste considerable.

12.2. Fuentes de financiación:

El proyecto se financiará completamente con mis propios recursos y no se planea buscar financiación externa, ya que se trata de un proyecto personal y académico. Los costos asociados a herramientas de desarrollo y servidores serán asumidos sin la necesidad de inversión externa, y todo el trabajo se realizará con tecnologías de código abierto y utilizando mi infraestructura local (PC personal).

12.3. Proyección de ingresos:

El proyecto no tiene como objetivo generar ingresos directos en esta etapa, ya que es un proyecto académico y personal. Sin embargo, si el juego se lanza comercialmente en el futuro, se podrían explorar modelos de monetización como la venta directa, micropagos dentro del juego o la publicidad dentro del juego. Estas decisiones se tomarán una vez que el proyecto esté más avanzado y se pueda evaluar su viabilidad económica.

12.4. Plan de sostenibilidad:

Dado que el proyecto se basa en herramientas gratuitas y se desarrollará en un entorno local, no se prevé un gasto significativo a largo plazo. Si el proyecto se amplía en el futuro, se podría migrar a un servidor dedicado o a servicios en la nube para garantizar la disponibilidad y escalabilidad del sistema. En este caso, se buscaría optimizar los recursos de infraestructura para mantener el proyecto viable sin generar costos excesivos.

12.5. Riesgos financieros:

El principal riesgo financiero en esta fase es la falta de recursos para adquirir servicios de infraestructura, lo cual podría limitar la capacidad de escalar el proyecto o hacerlo accesible a un

público más amplio. Sin embargo, el uso de infraestructura local y herramientas gratuitas minimiza estos riesgos. Si el proyecto se convierte en algo más grande, los costos asociados con el alojamiento en servidores externos o la expansión del alcance podrían representar un desafío.

12.6. Análisis de costes-beneficios:

En esta etapa, los costos se limitan principalmente a la infraestructura local (como la energía eléctrica y el uso del ordenador personal). Los beneficios incluyen la creación de un prototipo funcional que servirá como base para futuros desarrollos y aprendizajes personales. A medida que el proyecto se amplíe y si se considera una futura comercialización, el análisis de costes-beneficios se ampliará, evaluando los posibles ingresos frente a los costos de mantenimiento, promoción y servidores.

En resumen, el proyecto está siendo financiado de manera personal y no se prevé un modelo de ingresos inmediato. Los beneficios son principalmente de carácter formativo y académico, mientras que los costos son mínimos gracias al uso de tecnologías gratuitas y la infraestructura personal.

13. Conclusiones y valoración personal

13.1. Conclusiones:

El desarrollo de este sistema de Leaderboard en Unity, con un backend en Flask y una base de datos en PostgreSQL, ha permitido la implementación de un sistema robusto y escalable para gestionar las puntuaciones de los jugadores. Aunque en esta fase el proyecto no busca generar ingresos directos ni ser lanzado comercialmente, ha sido una excelente oportunidad para aplicar y consolidar mis conocimientos en desarrollo de software, integración de bases de datos y arquitectura de aplicaciones.

El sistema de Leaderboard ha sido diseñado con principios de eficiencia y modularidad, lo que facilita futuras ampliaciones y adaptaciones. A lo largo del desarrollo, se han resuelto diversos desafíos técnicos como la creación de un sistema de amistades, la gestión de puntuaciones duplicadas, la optimización de las consultas a la base de datos mediante el uso de SQLAlchemy y la implementación de una API para la interacción entre el frontend y el backend. Estos aspectos son cruciales para asegurar la integridad de los datos y la experiencia del usuario en cualquier sistema que maneje puntuaciones o clasificaciones.

Además, la implementación de una infraestructura local como servidor en mi PC, junto con la configuración de una IP estática, ha permitido avanzar con el desarrollo de forma eficiente, a pesar de no contar con un presupuesto externo. Esto ha demostrado que, con herramientas adecuadas y planificación, es posible llevar a cabo un proyecto técnico de calidad sin necesidad de grandes inversiones.

En el futuro, si el proyecto se amplía o se convierte en un producto comercial, se podrían considerar mejoras como la escalabilidad de la infraestructura, la inclusión de medidas de seguridad adicionales y el desarrollo de un modelo de monetización viable.

13.2. Valoración personal:

Este proyecto ha sido una valiosa experiencia tanto en el ámbito técnico como personal. A nivel profesional, me ha permitido consolidar mis conocimientos en desarrollo de Apis, aplicaciones multiplataforma y backend, trabajando con herramientas y tecnologías que son clave en la industria del software actual. La integración de Unity con Flask y PostgreSQL me ha brindado una comprensión más profunda sobre cómo desarrollar aplicaciones interactivas y manejar bases de datos de manera eficiente y segura.

A nivel personal, me ha dado la oportunidad de trabajar en un proyecto a largo plazo, lo que me ha permitido mejorar mis habilidades de planificación, resolución de problemas y toma de decisiones. El hecho de que el proyecto sea de carácter académico también me ha impulsado a reflexionar sobre las implicaciones que tendría en un contexto comercial, lo que me ha permitido pensar en aspectos como la gestión económica, la sostenibilidad y la viabilidad a largo plazo.

En cuanto a los desafíos que he enfrentado, El primero la creación de un sistema de amistades debido a que no sabia como debia crear la relación exactamente. Finalmente se opto por la opción mas optima que consiste en una tabla amistades que contiene las relaciones de manera unidireccional y un campo estado, por otra parte, tambien esta la gestión de puntuaciones duplicadas porque no queríamos mas que una puntuación por jugador y no se admitía nada mas que la puntuación mas alta

Si se quisiese desplegar la api en red la falta de recursos financieros puede ser un obstáculo, por eso he indagado en como podría hacerse mediante el uso de infraestructura local. Esto me ha demostrado que es posible realizar proyectos de gran calidad con recursos limitados, siempre que se cuente con una buena planificación y una visión clara de los objetivos a alcanzar.

En resumen, este proyecto ha sido un éxito en términos de aprendizaje y desarrollo técnico, y me ha dejado con una mayor confianza para abordar proyectos más grandes en el futuro. Estoy satisfecho con todos los avances realizados hasta ahora (como el aprendizaje en python que he obtenido, la integración de la funcionalidad para la gestión de amigos y de privacidad que tenia su dificultad, la inclusión de informes con jaspersoft y también de un servicio mail para recuperar la contraseña, la gestión de autenticación en la api mediante token jwt y ademas cors para limitar los orígenes de otros dominios si hiciera falta) así como estoy satisfecho también con el potencial que tiene el sistema, ya que proporciona una base sólida sobre la cual seguir creciendo y mejorando.

14. Bibliografia

Bibliografía que incluye todas las fuentes que he consultado para el aprendizaje de las tecnologías y herramientas utilizadas

1. Flask (Desarrollo Web con Python):

- Grinberg, M. (2018). *Flask web development: developing web applications with Python*. O'Reilly Media, Inc.

2. PostgreSQL (Sistema de Gestión de Bases de Datos):

- PostgreSQL Global Development Group. (2023). *PostgreSQL Documentation*. Recuperado de <https://www.postgresql.org/docs/>

3. Unity (Motor de Desarrollo de Videojuegos):

- Unity Technologies. (2023). *Unity User Manual*. Recuperado de <https://docs.unity3d.com/Manual/UnityManual.html>

4. JSON Web Tokens (JWT) (Autenticación y Autorización):

- Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT) (RFC 7519)*. Internet Engineering Task Force (IETF). Recuperado de <https://datatracker.ietf.org/doc/html/rfc7519>

5. JasperReports (Generación de Informes):

- TIBCO Software Inc. (2023). *JasperReports Ultimate Guide*. Recuperado de <https://jasperreports.sourceforge.net/JasperReports-Ultimate-Guide-3.pdf>

15. Anexos

Anexo A: Diagramas del Sistema

- **A.1. Diagrama de Arquitectura General:** Representación visual de la arquitectura del sistema, mostrando la interacción entre el cliente (Unity), la API (Flask) y la base de datos (PostgreSQL).
- **A.2. Diagrama de Clases:** Detalle de las clases principales utilizadas en el desarrollo del cliente en Unity y su relación con la API.
- **A.3. Diagrama Entidad Relación**
- **A.4. Modelo Relacional**
- **A.5. Modelo Normalizado**

Anexo B: Especificaciones Técnicas

- **B.1. Configuración del Servidor Flask:** Detalles sobre la configuración del entorno de desarrollo, dependencias y estructura del proyecto Flask.
- **B.2. Estructura de la Base de Datos PostgreSQL:** Descripción de las tablas, relaciones y restricciones definidas en la base de datos para gestionar las puntuaciones y perfiles de usuarios.

- **B.3. Detalles de Implementación de JWT para Autenticación:** Explicación de cómo se implementó la autenticación basada en JSON Web Tokens en la API.

Anexo C: Código Fuente Relevante

- **C.1. Ejemplos de Endpoints de la API:** Listado de los endpoints más importantes con su respectiva funcionalidad y ejemplos de uso.
- **C.2. Fragmentos de Código del Cliente en Unity:** Secciones clave del código en C# que muestran cómo se realiza la comunicación con la API.

Anexo D: Manual de Usuario

- **D.1. Guía para Desarrolladores:** Instrucciones sobre cómo integrar el sistema de Leaderboard en otros proyectos desarrollados con Unity.
- **D.2. Guía para Jugadores:** Descripción de cómo los usuarios pueden interactuar con el sistema de puntuaciones, gestionar su perfil, privacidad y amigos.

Anexo E: Documentación Adicional

- **E.1. Implementación del Servicio de Correo Electrónico:** Descripción de cómo se integró el servicio de correo para la recuperación de contraseñas, incluyendo dependencias y configuración.
- **E.2. Configuración de CORS:** Detalles sobre la configuración de Cross-Origin Resource Sharing en la API para permitir o restringir solicitudes desde diferentes dominios.
- **E.3 Generación de informes:** muestra de los informes generados

Avances de ultima hora:

- Añadir texto y voz al diálogo de nivel 2.
- Mejorar mapas.
- crear funcionalidad borrar usuario
- crear funcionalidad borrar amistad
- Añadir AK47 al juego.
- Añadir sistema de guardando (Usando PlayerPrefs de forma temporal)
- Añadir sistema de oleadas
- Añadir sistema de oleadas infinito con tiempo para ganar (y teniendo que poner una bomba para ganar)
- añadir mecánicas de estamina al jugador
- Arreglar sonido de disparos.
- Arreglar bug de recargar armas. (cambiar de arma mientras se recarga el arma anterior hace que este que estabamos recargando quede inutilizable)
-solucion no permitir cambiar de arma mientras se recarga
- Arreglar error juego bug graficos al intentar seguir la segunda cámara a un objeto de la primera cuando esta esta inactiva, solución temporal meter el objeto (Punto Mira) en el player
-solucion: quitar el script que sigue a un objeto de FirstPersonCamera2 y añadir el objeto FirstPersonCamera2 dentro de FirstPersonCamera
- Integrar token de refresco.
- Generar un nuevo archivo para exportar el leaderboard y generar uno para exportar el menu de pausa
- Añadir lo necesario para crear build en Android

Avances Futuros:

- Añadir seguridad SSL y HTTPS a la API.
- Modificar sistema de guardado para que use un archivo que genere automaticamente en vez de los PlayerPrefs
- Añadir Funcion de Autoguardado Opcional y configurable desde menu de opciones
- añadir mas armas
- Añadir funcion remember password al login